

Data Layer Design

Data Layer Design

- Introduction
- Relational data base technology
- Strategies for managing persistence
- Automatic generation of persistence
 - Data Mapper pattern
 - Variant: query operations in the Data layer
- Direct design of persistence
 - Direct design of operations
 - Domain layer controller with plain logic
 - Domain layer controller as a pure intermediary
- Example
- Design of the Domain and Data layers, summary
- References

Introduction

What is, and how do we get, persistence?

- **Persistence:**
 - ability that most software systems require for storing and obtaining data using a permanent storage system
- **Objects can be made persistent by using:**
 - Object-oriented data bases
 - Relational data bases
 - Object-relational data bases
 - XML files
 - etc.

Introduction

How does Data Base technology influence design?

Technological dependence:

- Properties that have to be satisfied (non-functional requirements)
- Technological resources available
 - Programming language paradigm
 - ***Data Base Management System (DBMS) paradigm***

Determine:

- The architecture of the software system and the patterns to be applied

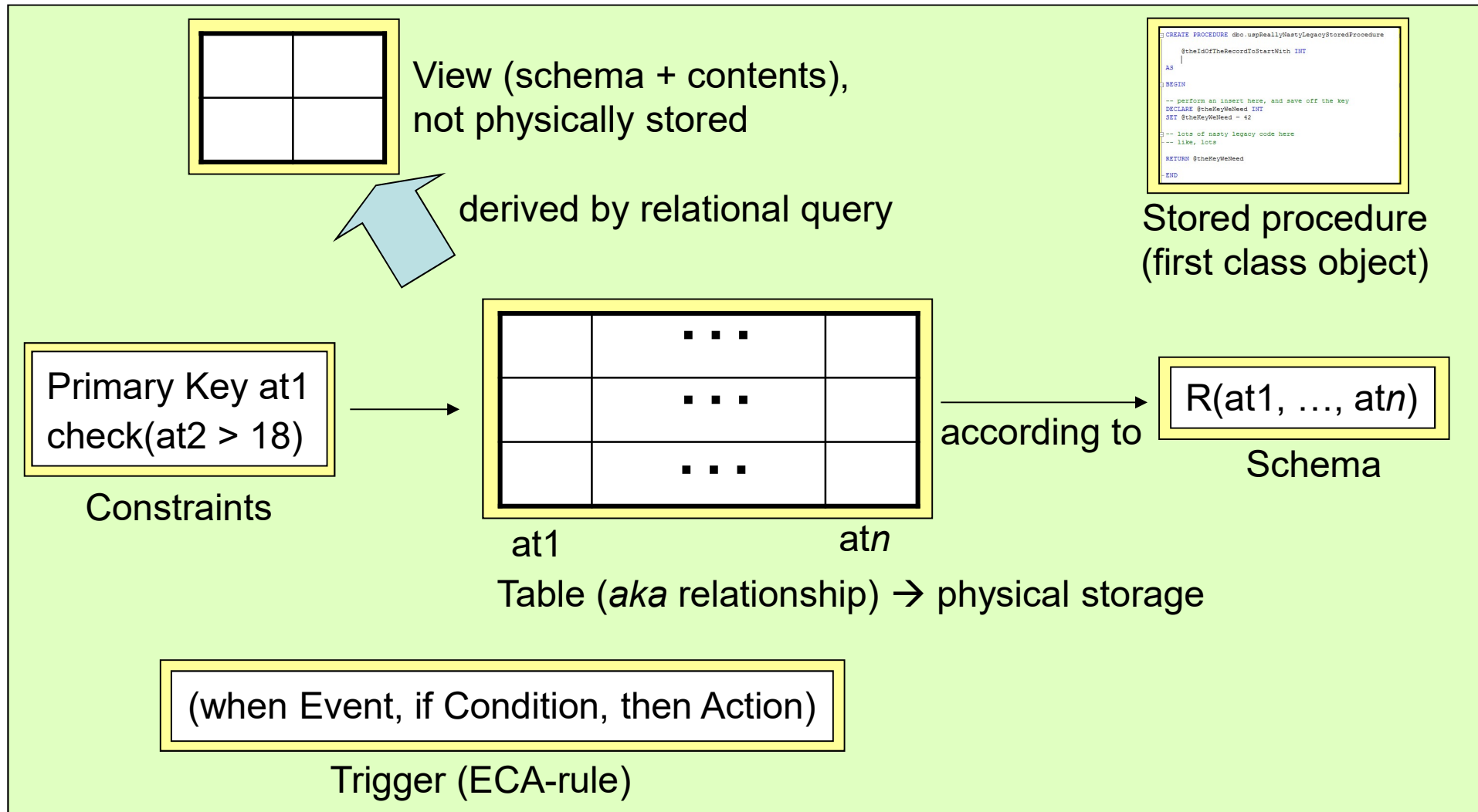


Software design depends on the DBMS type used

Along this course we focus on the study
of persistence using relational DBMS

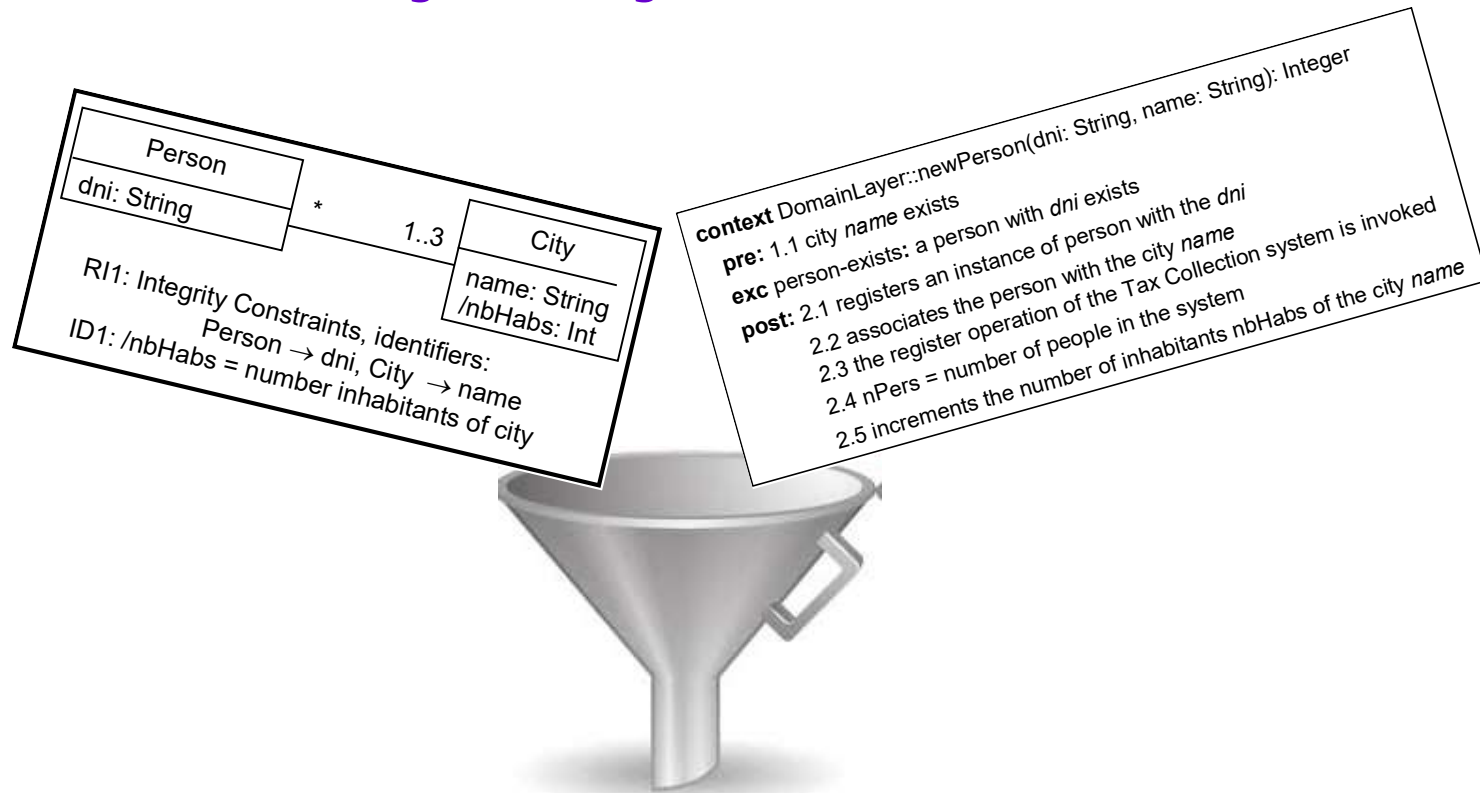
Relational Data Base Technology:

Data and Process components



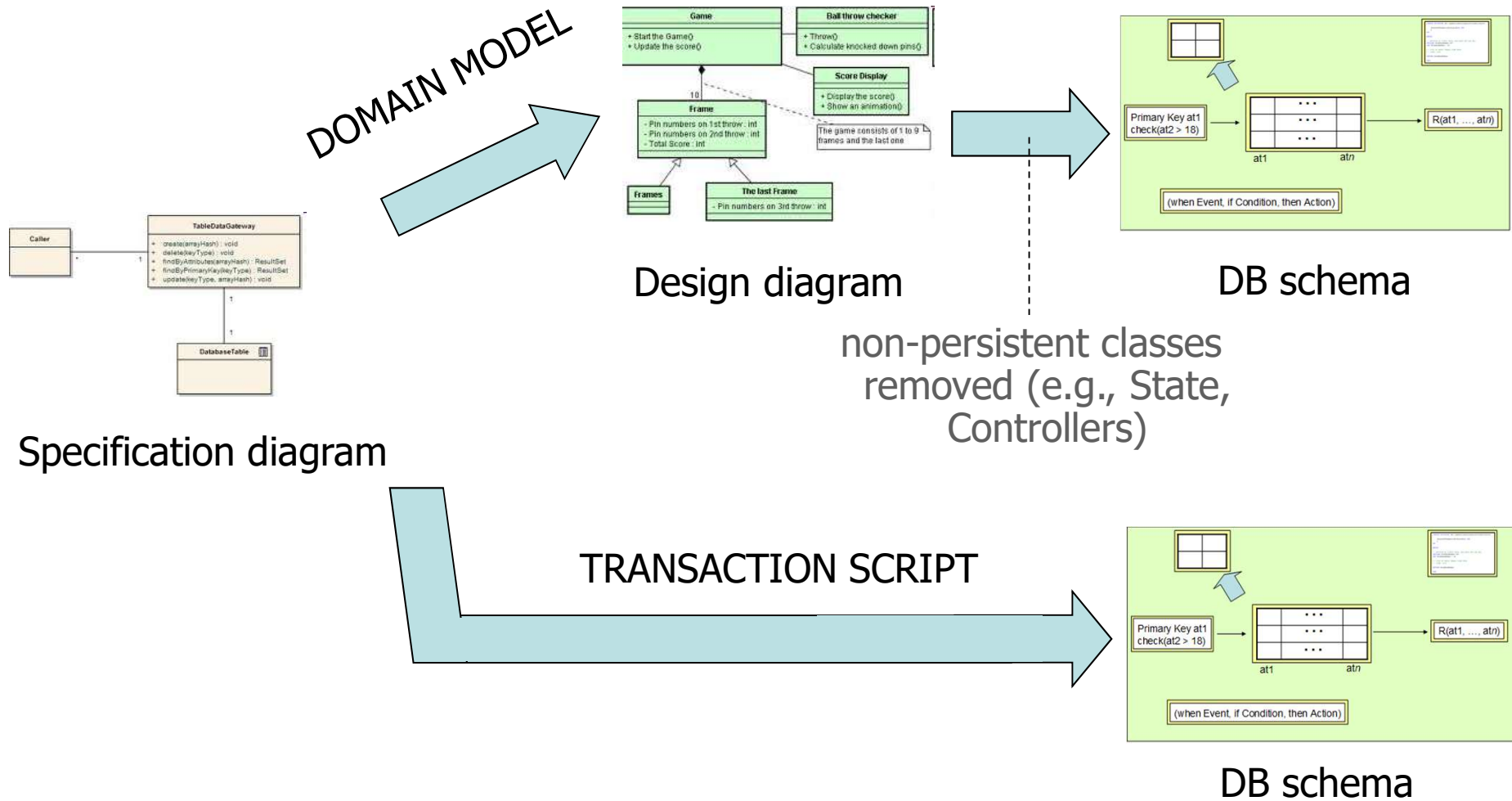
Relational Data Base Technology:

Logical Design of the DB Schema



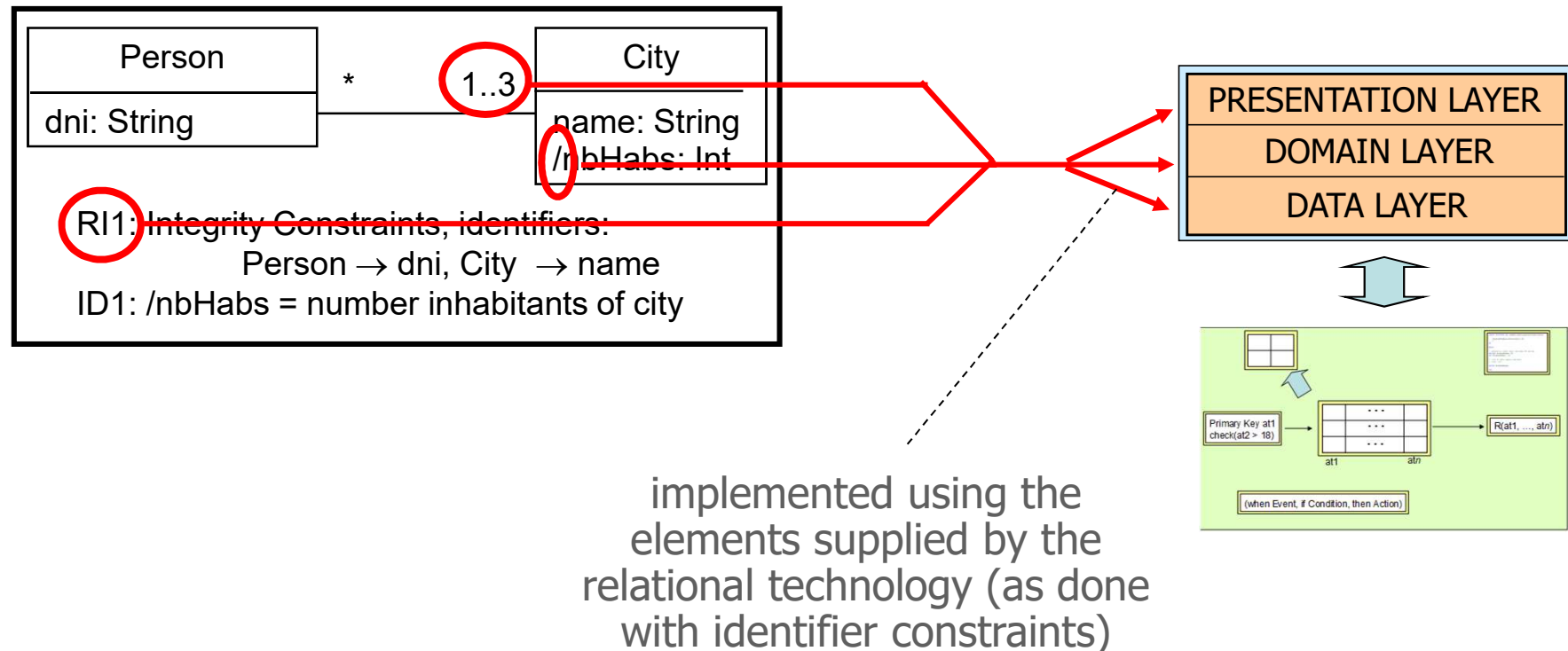
Relational Data Base Technology:

Logical Design of the DB Schema – Translation of the Class Diagram



Relational Data Base Technology:

Logical Design of the DB Schema – Translation of other Responsibilities



Relational Data Base Technology:

Logical Design of the DB Schema – Translation of other Responsibilities

context DomainLayer::newPerson(dni: String, name: String): Integer

pre: 1.1 city *name* exists

exc ~~person-exists: a person with *dni* exists~~

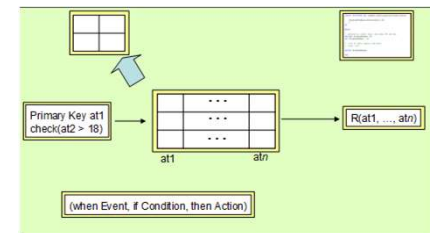
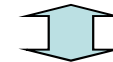
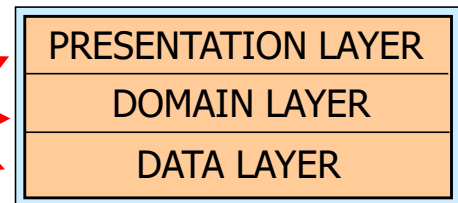
post: 2.1 registers an instance of person with the *dni*

2.2 associates the person with the city *name*

2.3 the register operation of the Tax Collection system is invoked

2.4 nPers = number of people in the system

2.5 increments the number of inhabitants nbHabs of the city *name*



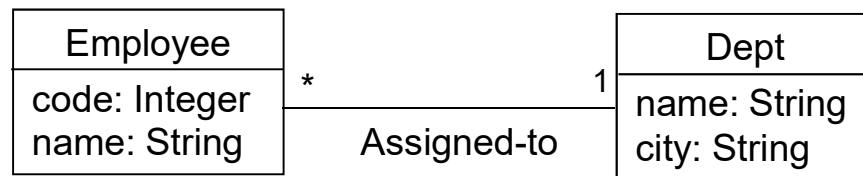
implemented using the
elements supplied by the
relational technology (as done
with identifier constraints)

Logical Design of the DB Schema – Not Covered in Relational Technology



Relational Data Base Technology:

Logical Design of the DB, binary associations



Is translated into:

Dept (name-d, city)

Employee (code, name-emp, name-d)



Is translated into:

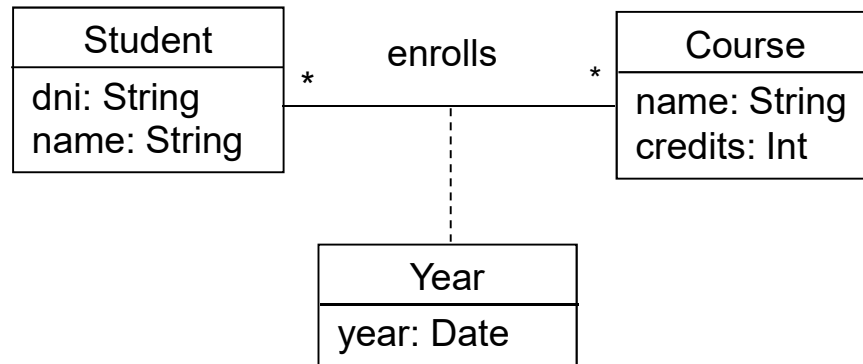
Interval (date, init)

Court (num)

Maintenance (num-court, date, init)

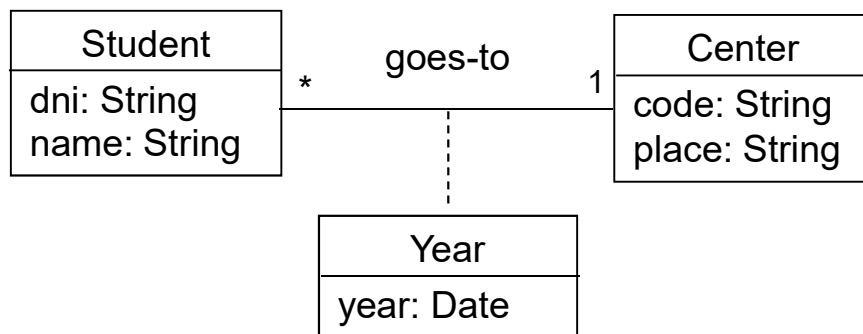
Relational Data Base Technology:

Logical Design of the DB, class associations



Is translated into:
Student (dni, name)
Course (name, credits)
Year(dni, name, year)

In the unlikely event that some class association has a role with multiplicity 1 (or 0..1), the table is created anyway:

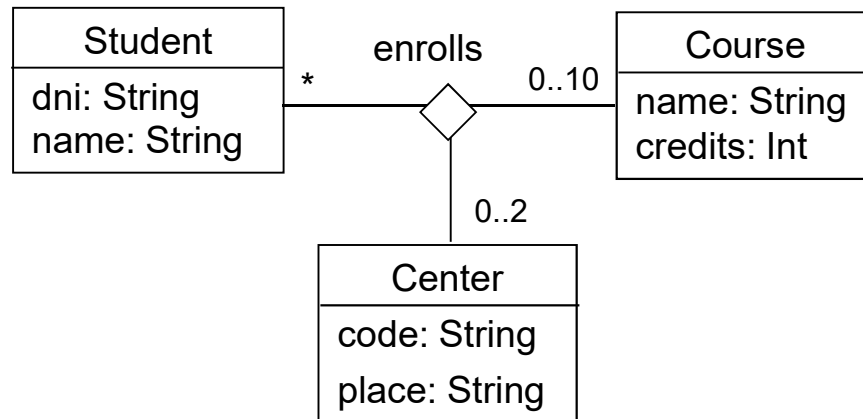


Is translated into:
Student (dni, name, codeCenter)
Center (code, place)
Year(dni, code, year)

Relational Data Base Technology:

Logical Design of the DB, n-ary associations, $n > 2$

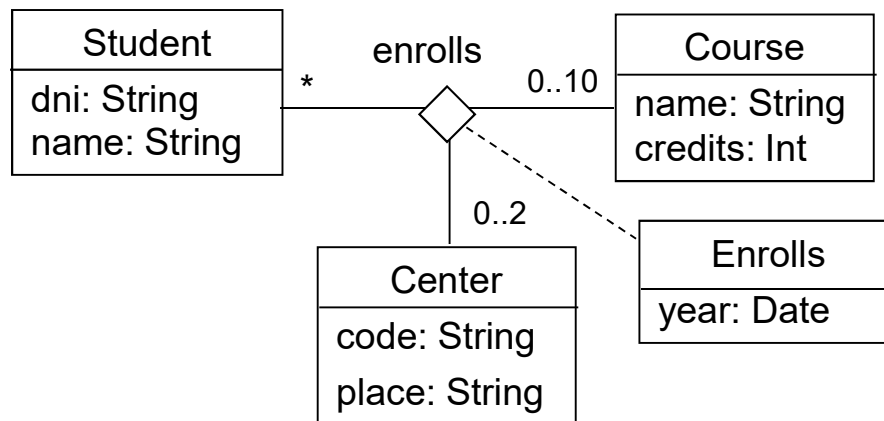
The treatment does not depend on multiplicities:



Is translated into:

Student (dni, name)
Course (name, credits)
Center (code, place)
enrolls (dni, name, code)

If it is class association, the treatment is the same:



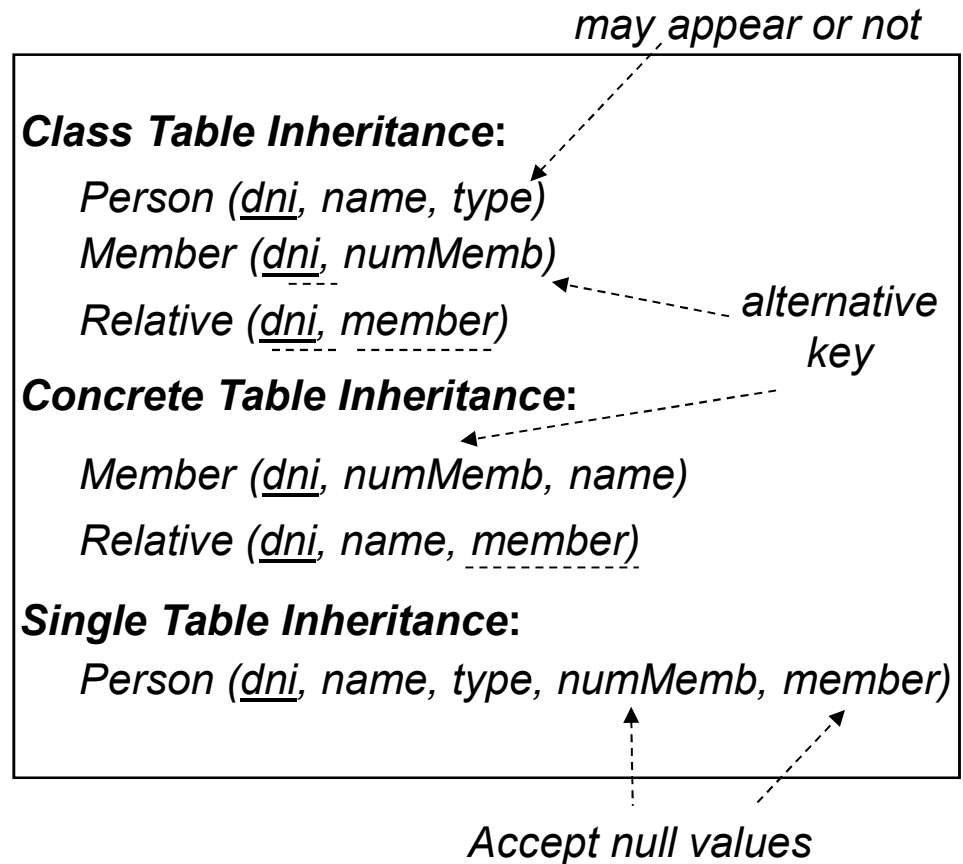
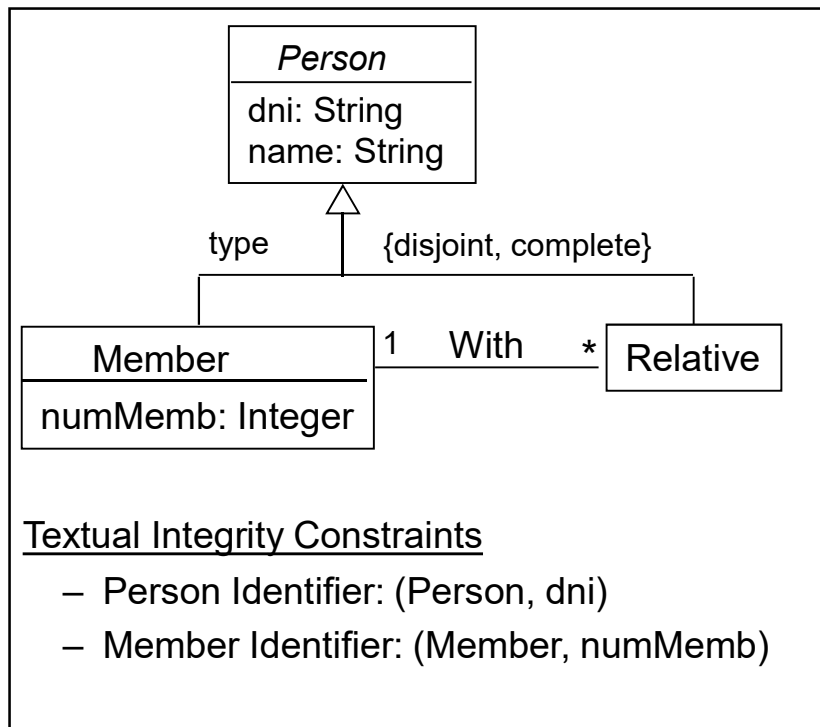
Is translated into:

Student (dni, name)
Course (name, credits)
Center (code, place)
enrolls (dni, name, code, year)

Relational Data Base Technology:

Logical Design of the DB, specialization hierarchies (1)

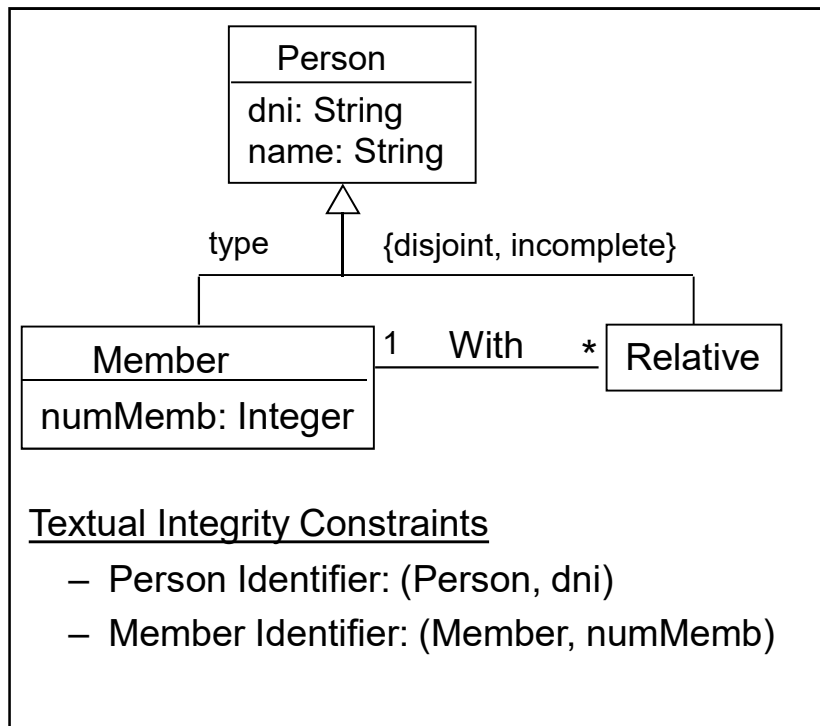
The translation depends on whether the hierarchy is collapsed or not



Relational Data Base Technology:

Logical Design of the DB, specialization hierarchies (1, variant)

The translation depends on whether the hierarchy is collapsed or not



Class Table Inheritance:

Person (dni, name, type) *may appear or not*

Member (dni, numMemb)

Relative (dni, member) *alternative key*

Concrete Table Inheritance:

Person (dni, name)

Member (dni, numMemb, name)

Relative (dni, name, member)

Single Table Inheritance:

Person (dni, name, type, numMemb, member) *Accept null values*

Relational Data Base Technology:

Logical Design of the DB, specialization hierarchies (2)

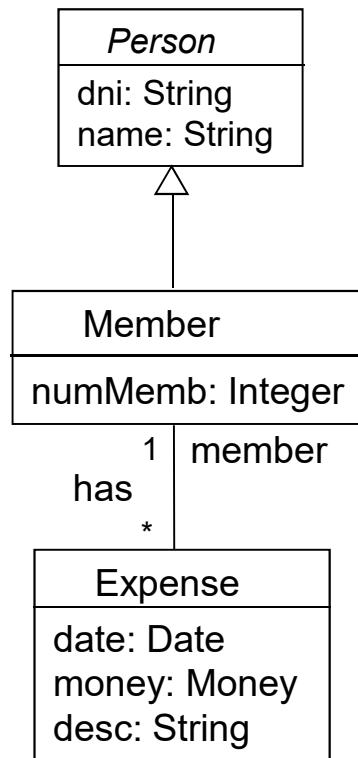
Strategy	Benefits	Drawbacks
<i>Class Table Inheritance</i>	Simple Changeable	Not much efficient (multiple accesses per object)
<i>Concrete Table Inheritance</i>	Efficient (one access per object)	Not much changeable (propagation of changes into abstract classes)
<i>Single Table Inheritance</i>	Efficient (one access per object) Changeable	Loss of space (but the DB may help)

There is not “the best” strategy. Talk to your Data Base Administrator!

Relational Data Base Technology:

Logical Design of the DB, classes without external keys

- It is necessary to add an “artificial” external key
 - Usually it will be a key maintained by the system itself



Textual Integrity Constraints

- Keys: (Person, dni), (Member, numMemb)

The table Expense looks like:

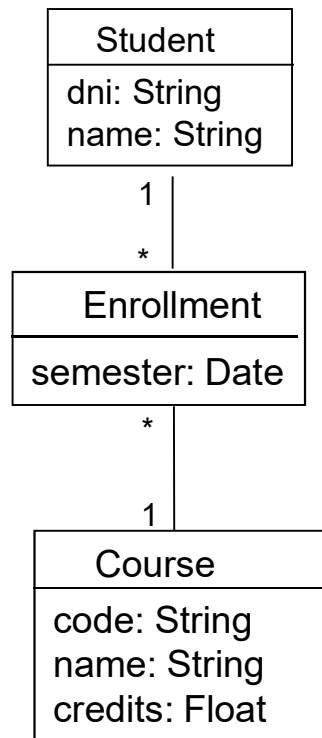
Expense(id-expense, date, payment, desc, member)

“artificial” identifier

Relational Data Base Technology:

Logical Design of the DB, classes with inconvenient keys

- Some keys are inconvenient to manage
 - The concept of key may change
 - The key is composed of several attributes



Textual Integrity Constraints

- Keys: (Student, dni), (Course, code)
- There cannot be more than one enrollment for a student and course a given semester

Non-functional requirements

- The system shall be prepared to future offering of courses to foreign students

Course(code, name, credits)

Student(id-student, dni, name)

Enrollment(id-enroll, id-student, code, semester)

Relational Data Base Technology:

Management of integrity constraints

- Relational DBMS provide diverse functionalities for dealing with integrity constraints:

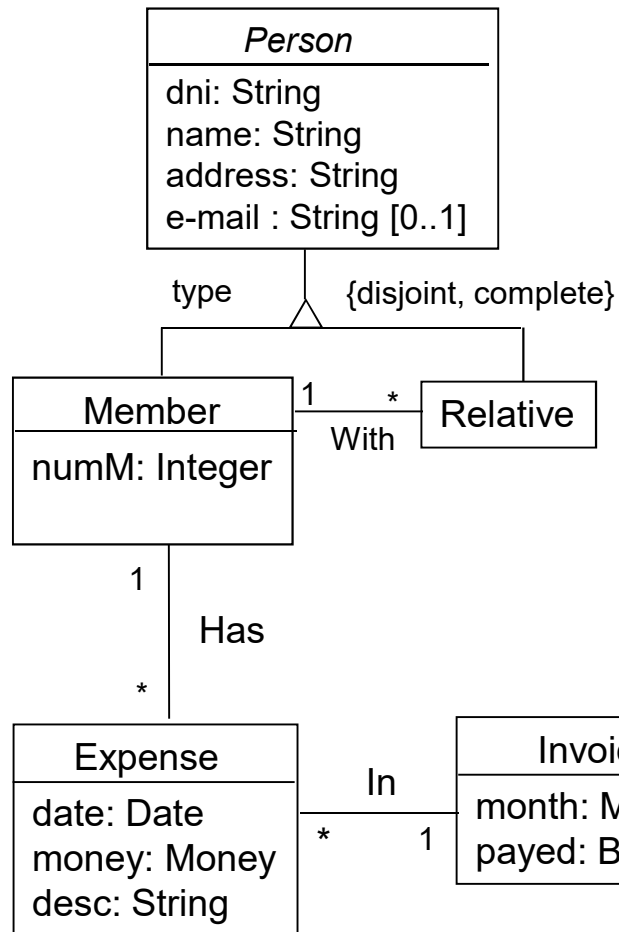
```
CREATE TABLE Invoices (  
    InvoiceNumber    INTEGER NOT NULL,  
    SupplierNumber  INTEGER NOT NULL,  
    Text            VARCHAR(4096),  
    CONSTRAINT invoice_pk PRIMARY KEY (InvoiceNumber),  
    CONSTRAINT inumber_value CHECK (InvoiceNumber > 0),  
    CONSTRAINT supplier_fk FOREIGN KEY (SupplierNumber)  
        REFERENCES Supplier(SupplierNumber)  
        ON UPDATE CASCADE ON DELETE RESTRICT )
```



The corresponding responsibilities may be directly assigned to the relational DBMS

Relational Data Base Technology:

Management of integrity constraints, example



Person (*dni*, *name*, *address*, *e-mail*)

Relative (*dni*, *member*)

Member (*dni*, *numM*)

Invoice (*dni*, *month*, *payed*)

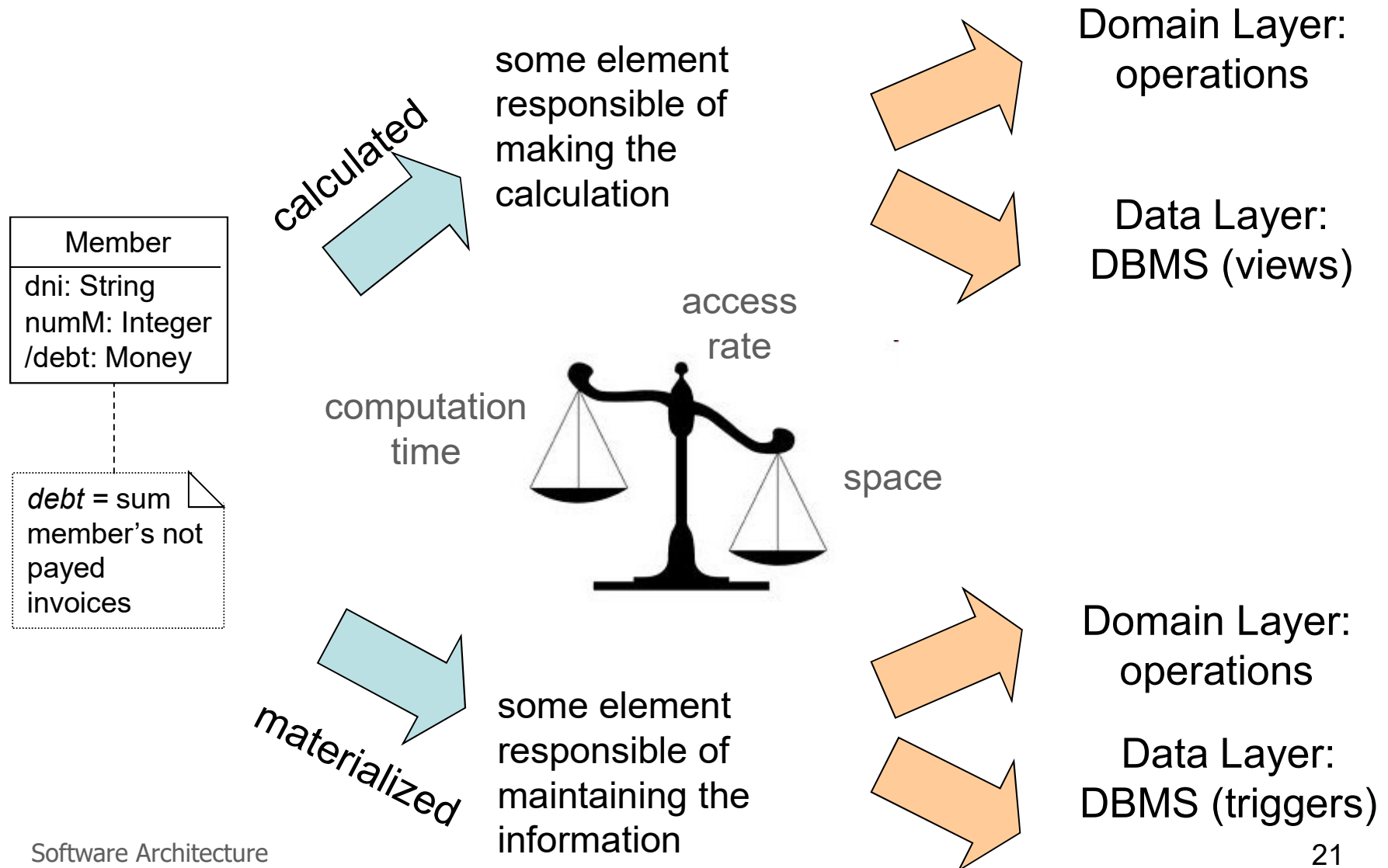
Expense (*id-expense*, *date*, *money*, *desc*, *member*, *month*)

Textual Integrity Constraints

- Identifiers: (Person, dni)
=> Primary key of *Person*: *dni*
- Identifiers: (Member, numM)
=> Primary key of *Member*: *dni* and alternative key, *numM*:
unique(numM)
- There cannot be two invoices for the same member and month
=> Primary key of *Invoice*: (*dni*, *month*)
- All the expenses of an invoice belong to the same month
=> Check of *Expense*: check(month(data) = month)

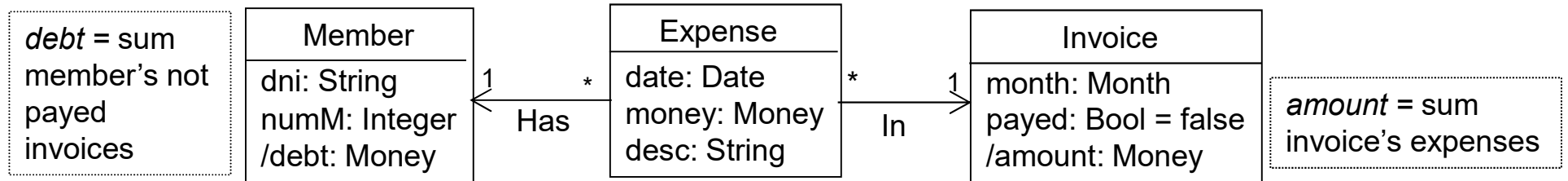
Relational Data Base Technology:

Dealing with derived information

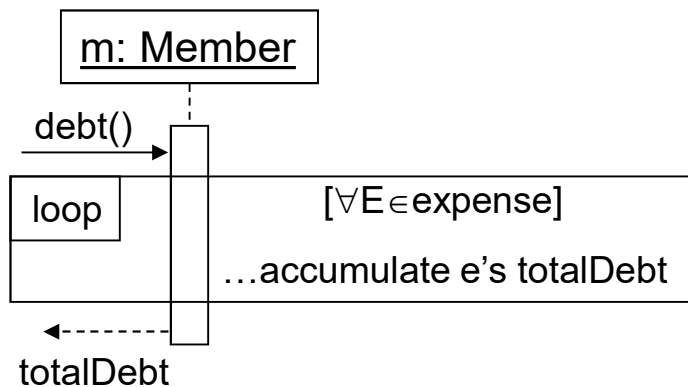
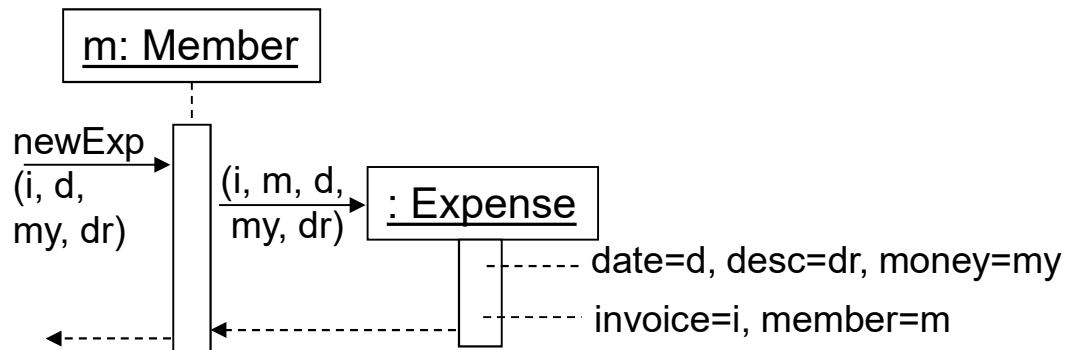


Relational Data Base Technology:

Dealing with derived information, calculated case domain layer's assignment



DOMAIN LAYER

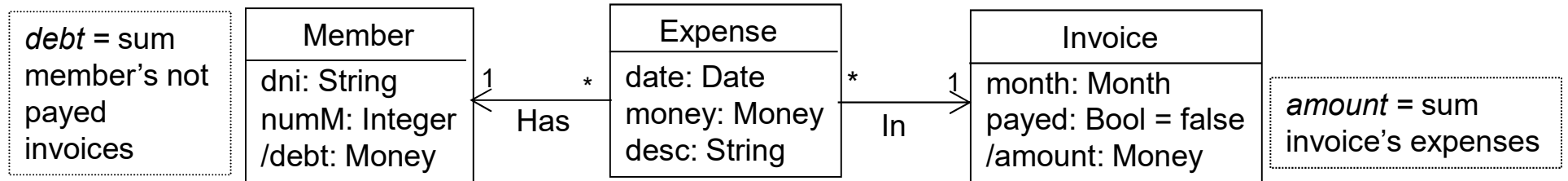


DATA LAYER

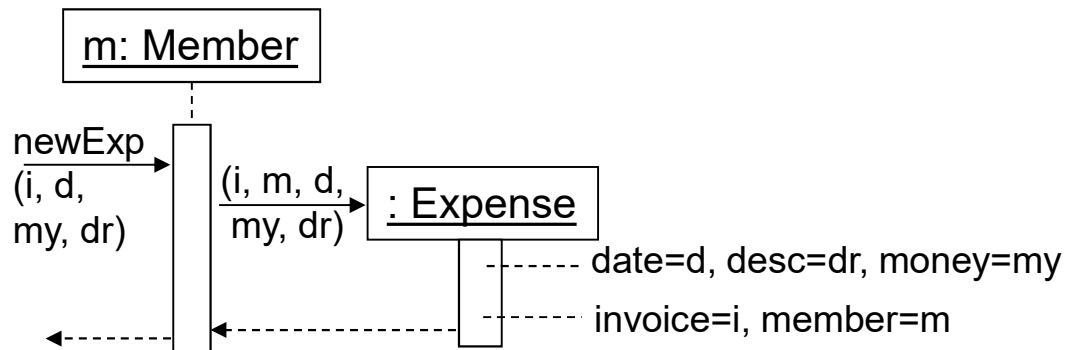
Member (dni, numM)
Invoice (dni, month, payed)
Expense (id-exp, date, money, desc,
 member, month)

Relational Data Base Technology:

Dealing with derived information, calculated case data layer's assignment with views



DOMAIN LAYER



DATA LAYER

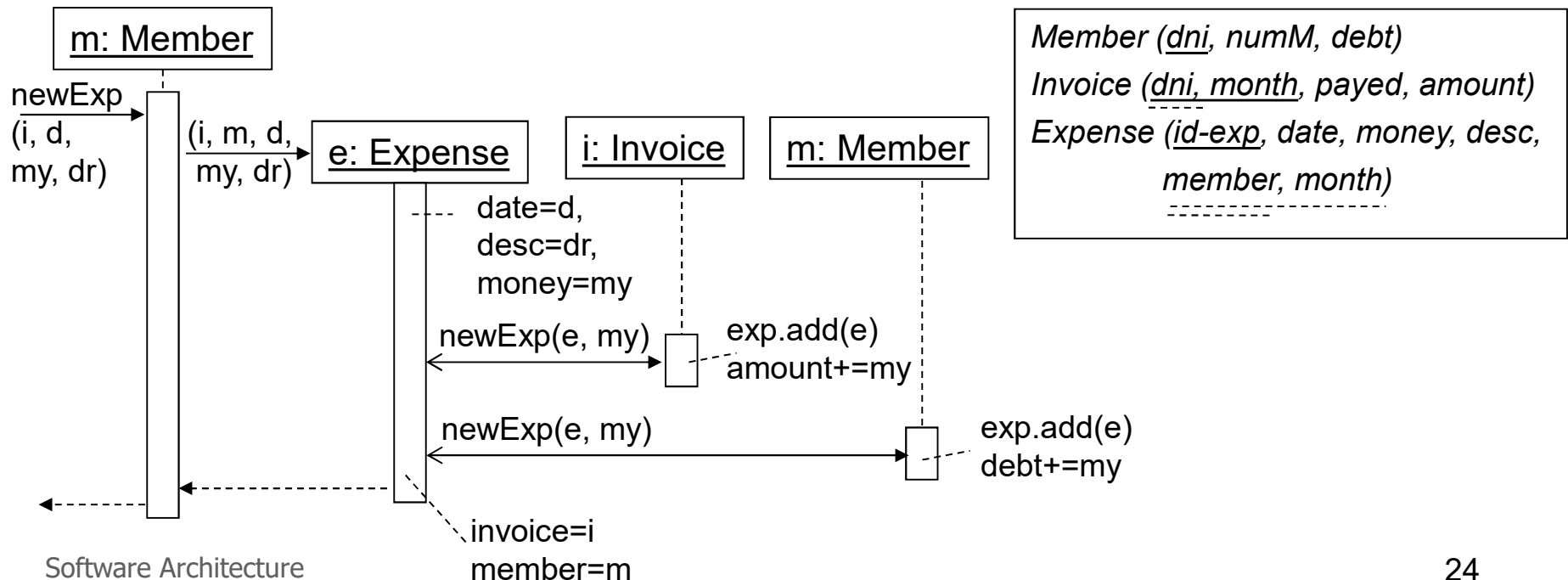
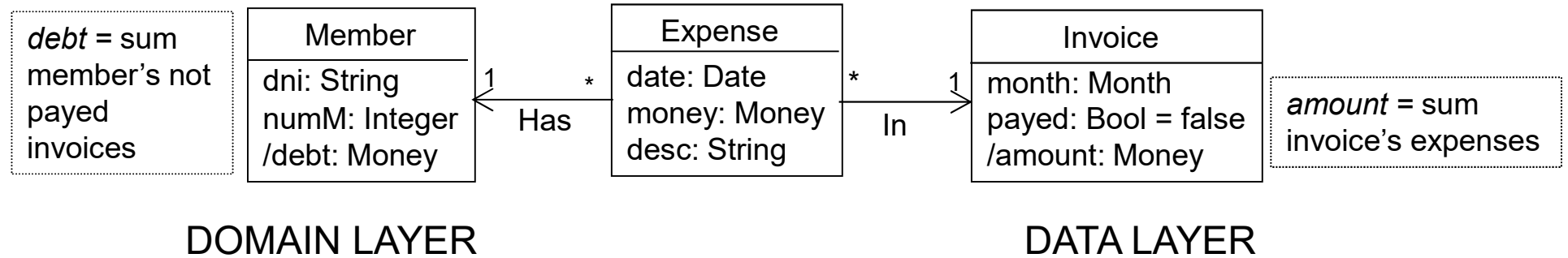
```

Member (dni, numM)
Invoice (dni, month, payed)
Expense (id-exp, date, money, desc,
        member, month)
        -----

CREATE VIEW  debt [dni, debt] AS
SELECT      s.dni, sum(c.money)
FROM        Member s, Expense c, Invoice r
WHERE       s.dni = c.member AND
            c.member = r.dni AND
            c.month = r.month AND
            r.payed='F'
GROUP BY   dni
  
```

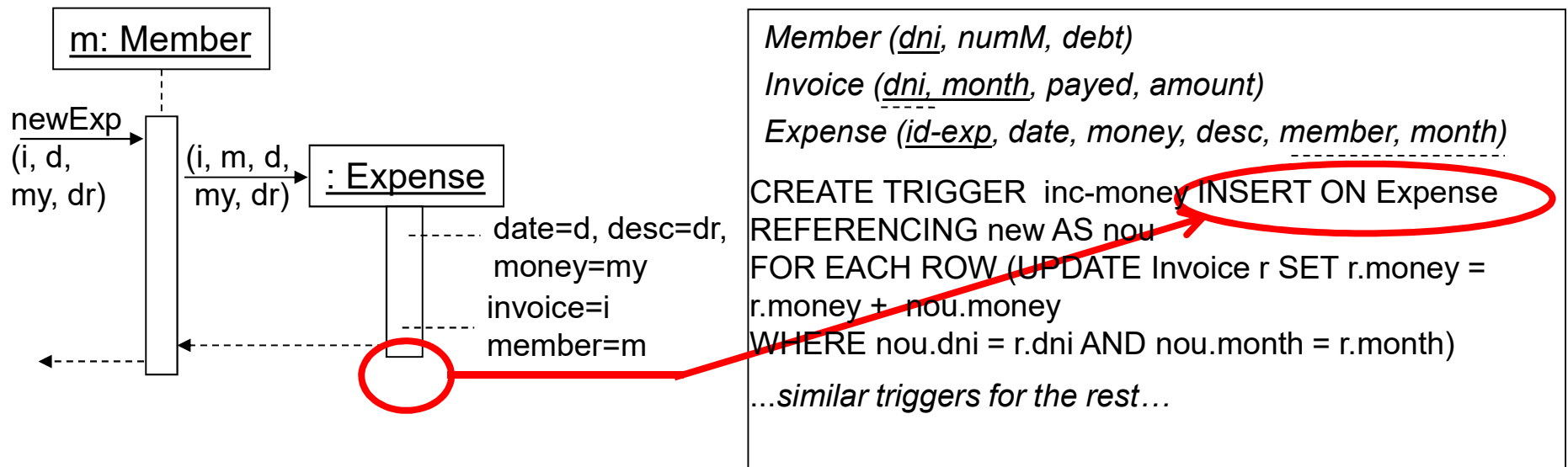
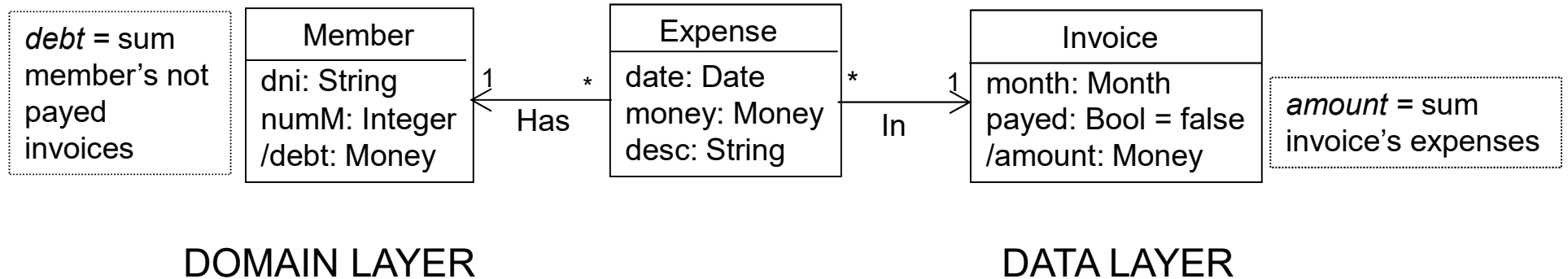
Relational Data Base Technology:

Dealing with derived information, materialized case domain layer's assignment



Relational Data Base Technology:

Dealing with derived information, materialized case data layer's assignment with triggers



Relational Data Base Technology:

Operation design, assigning responsibilities to the DBMS

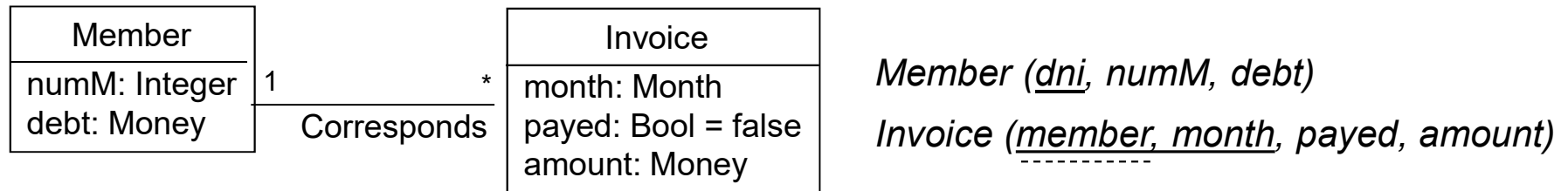
- Current specification languages do not allow defining active behaviour in conceptual schemas.
- On the contrary, relational DBMS do.
- Therefore, sometimes specification contracts define aspects that the DBMS may manage directly.



These responsibilities may be assigned to the DBMS

Relational Data Base Technology:

Operation design, assigning responsibilities to the DBMS



context DomainLayer::removeMember(dniM: Integer)

exc member-not-exists: there is no member with dniM

post 2.1: the Member with dniM is removed

2.2: All the invoices of that Member are removed

Assigned
to the
DBMS

```
CREATE TABLE Member (
  dni STRING PRIMARY KEY,
  ...)
```

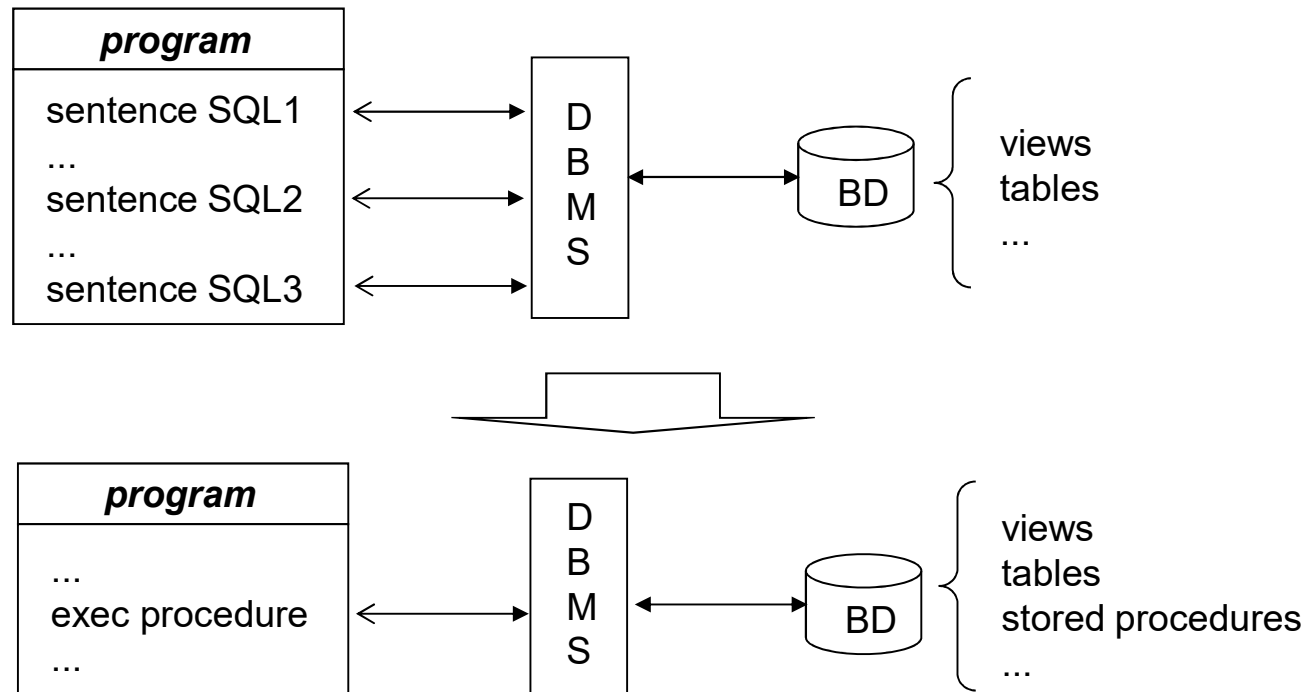
```
CREATE TABLE Invoice (
  member STRING,
  month MONTH,
  payed BOOLEAN,
  amount MONEY,
  PRIMARY KEY (member, month),
  FOREIGN KEY (member) references(Member) ON DELETE CASCADE)
```

**When removing a Member,
the DBMS automatically
removes its invoices**

Relational Data Base Technology:

Operation design, stored procedures

- They are used to:
 - Make application development simpler
 - Improve the DB performance
 - Control the operations that users execute against the DB
- May compromise portability



Relational Data Base Technology:

Operation design, stored procedures

<i>Member</i> (<u>dni</u> , numM, debt, <u>ba</u>)
<i>Invoice</i> (<u>member</u> , <u>month</u> , payed, amount)
<i>BankAccount</i> (<u>num-ba</u> , balance)

```
CREATE PROCEDURE PayInvoice (dniM, monthTramitation)
RETURNING INTEGER, CHAR(50);
DEFINE error-code INTEGER; ....
ON EXCEPTION SET error-code, error-miss; RETURN error-code, error-miss END EXCEPTION;

IF ((SELECT COUNT(*) FROM invoice WHERE month=monthTramitation AND member=dniM)=1) THEN
  LET amount, l-payd = (SELECT amount, payed FROM invoice WHERE month=monthTramitation AND member=dniM));
  IF ('Y' = l-payd ) THEN RAISE EXCEPTION 2, 'The invoice is already payed';
  ELIF LET balance, numba = (SELECT c.balance,c.num-ba FROM member s, bankaccount c WHERE s.ba=c.num-ba
                                and s.dni=dniM);
    IF balance < amount THEN RAISE EXCEPTION 3, 'The member has not balance enough';
    ELSE UPDATE invoice SET payed = 'Y' WHERE month=monthTramitation AND dni=dniM;
      UPDATE member s SET s.debt = s.debt-amount WHERE s.dni=dniM;
      UPDATE bankaccount c SET c.balance = c.balance-balance WHERE num-ba =c.num-ba ENDIF
ELSE RAISE EXCEPTION 1, 'The member has not invoices this month'; END IF;
RETURN 0,'PayedInvoice';
END PROCEDURE;
```

Strategies for managing persistence:

Automatic generation

- Automatic generator of persistence:
 - Software system that makes automatically the required translation for storing objects' data in secondary storage
 - Converts the updates made in main memory into updates in secondary storage
 - It requires the designer to specify the correspondence among the object-oriented class diagram and the data base schema
- Characteristics:
 - Persistence management is transparent to the designer
 - DBMS functionalities may not be fully exploited

It is used jointly with the Domain Model design pattern (as it has been done in chapter 6)

Strategies for managing persistence:

Direct design

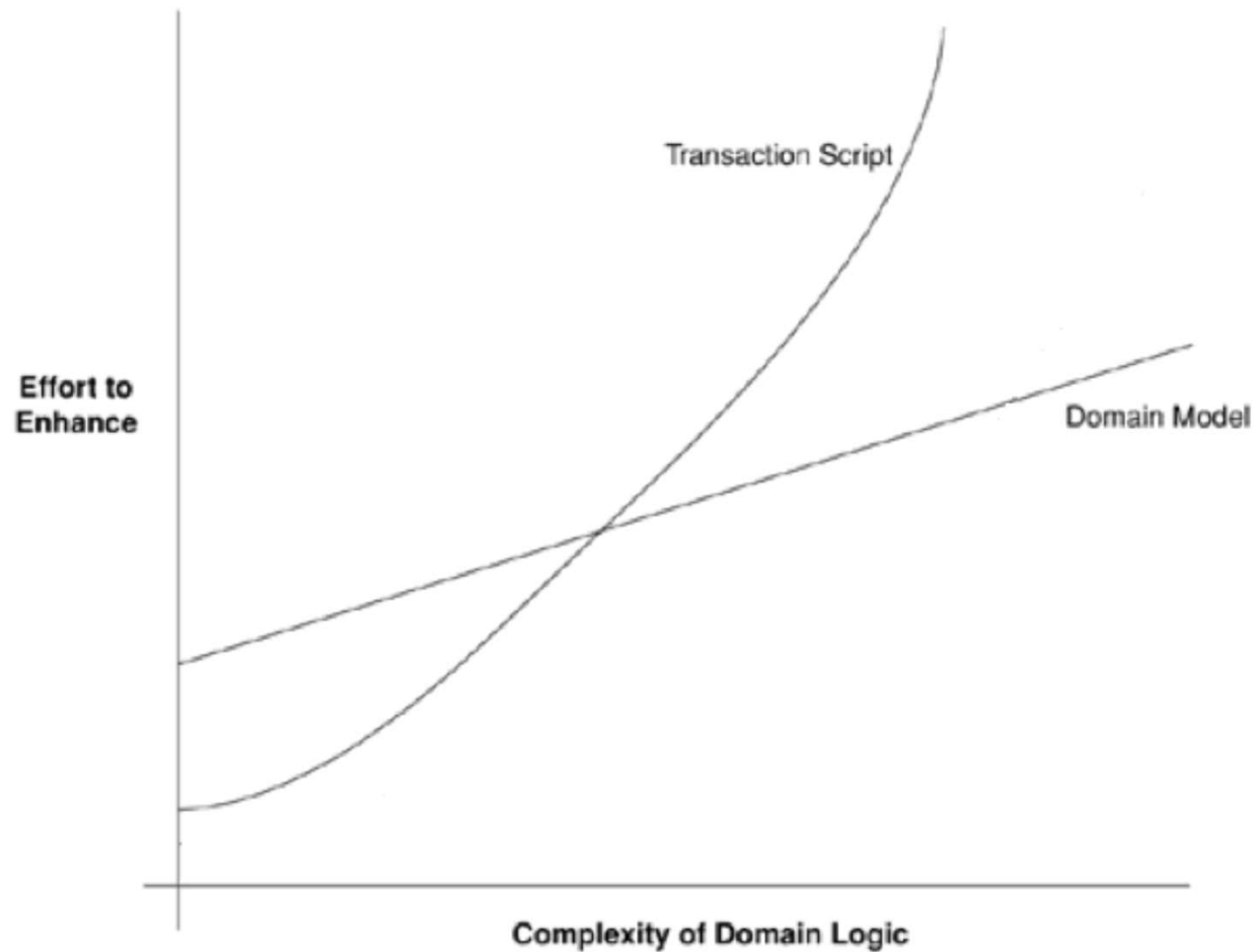
- Persistence direct design:
 - Software design is elaborated taking into account the DBMS that will be used when implementing
 - Therefore, the design will look different depending on whether we use an object-oriented DBMS, a relational one, etc.
- Characteristics:
 - The designer needs to know the DBMS in order to design the Data and Domain layers
 - The full functionalities of the DBMS may be exploited
 - The resulting design is similar for all the DBMS of the same family of DBMS

It is used jointly with the Transaction Script design pattern

Choosing the right pattern for domain layer

- The decision-making process will be mainly based on judging the complexity of the application's domain logic.
- Transaction Script for software systems with a lower complexity level and Domain Model for software systems where the business logic is more demanding.
- Transaction Script will definitely hit a wall much faster than Domain Model when the complexity of the business logic is increased by adding more features

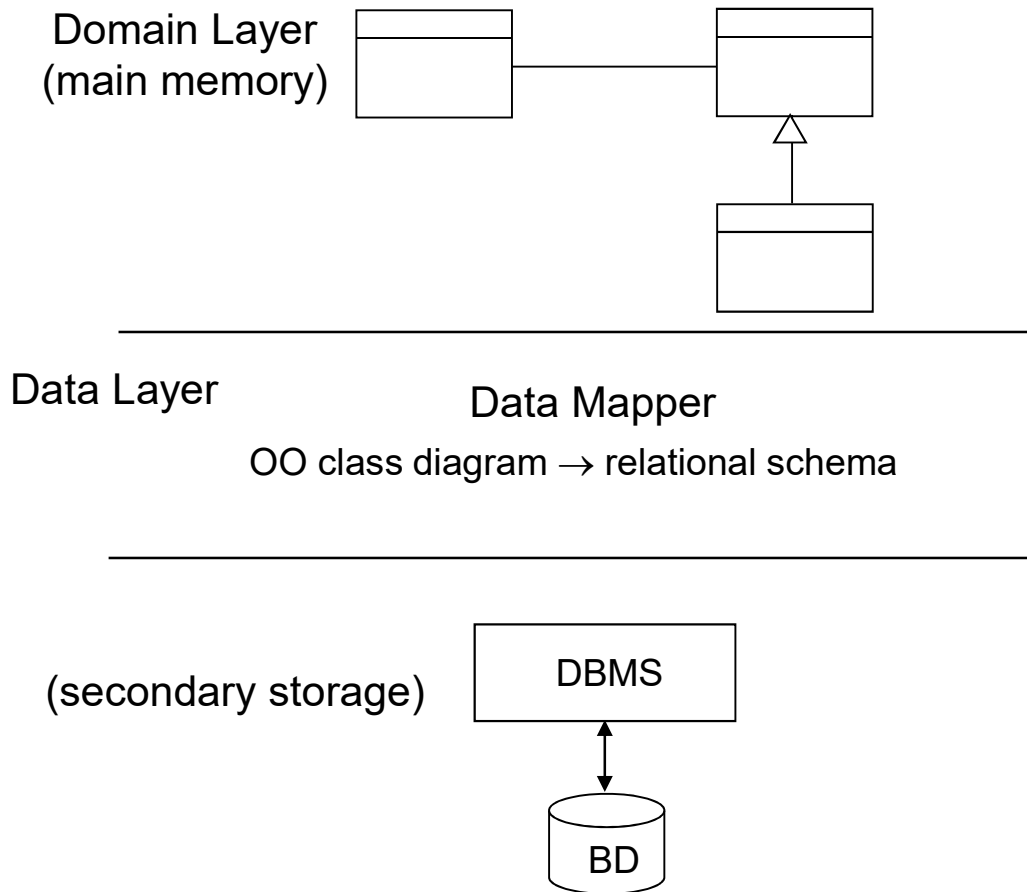
Choosing the right pattern for domain layer



Relationship between effort and complexity for different domain logic architecture design patterns.
Source: [Patterns of Enterprise Application Architecture](#) by Martin Fowler

Automatic generation of persistence:

Data Mapper design pattern



- The designer defines the correspondence among the normalized class diagram and the relational schema
- The Data Mapper maintains the correspondence between the objects and the tables.
- The Data Mapper knows (almost) automatically the updates made at the objects and knows how to propagate them into the tables.
- At the design stage, the sequence diagrams do not change
- Different products implement the pattern in different ways:
 - Hibernate, TopLink, ...

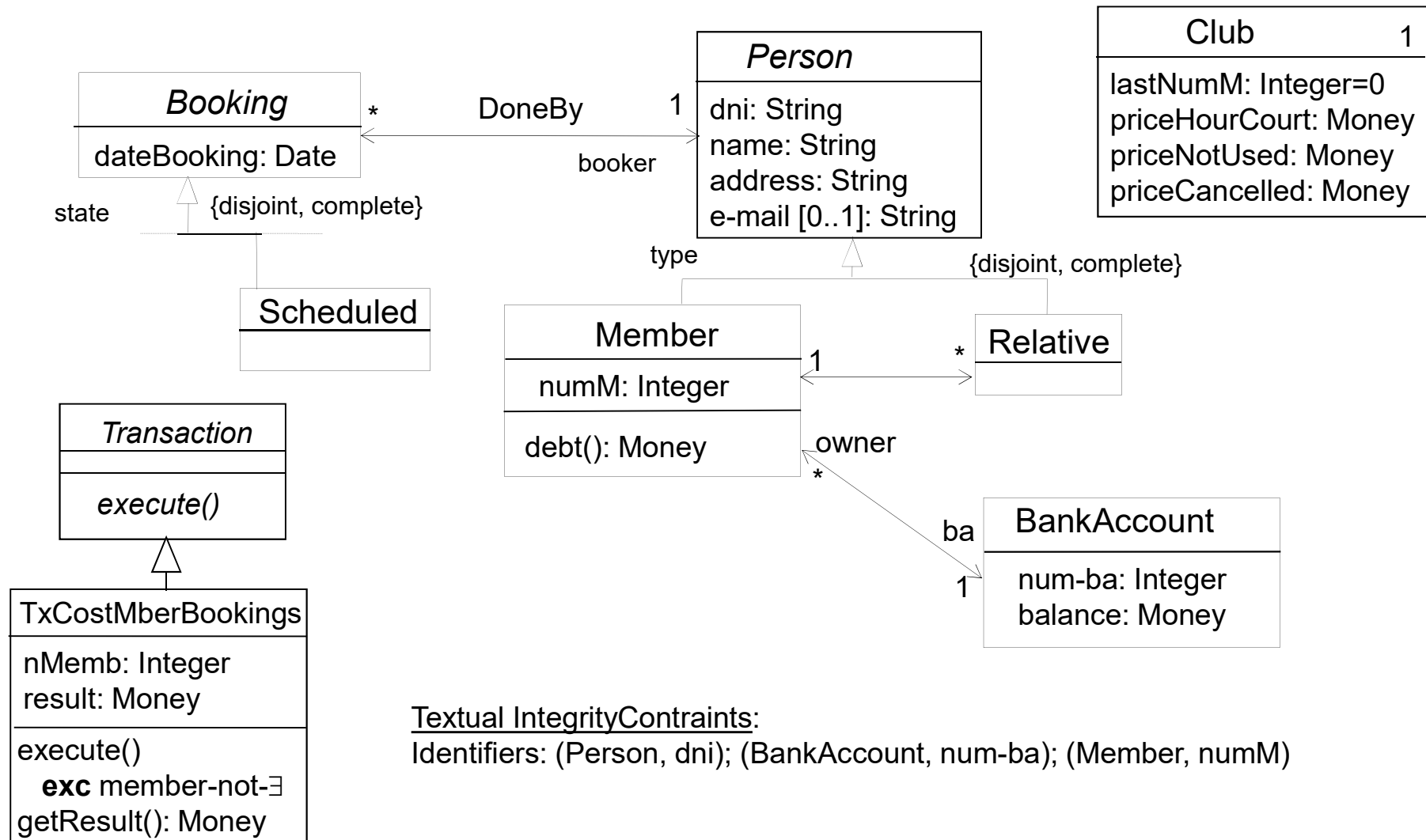
Automatic generation of persistence:

Arbitrary query operations on the Data layer

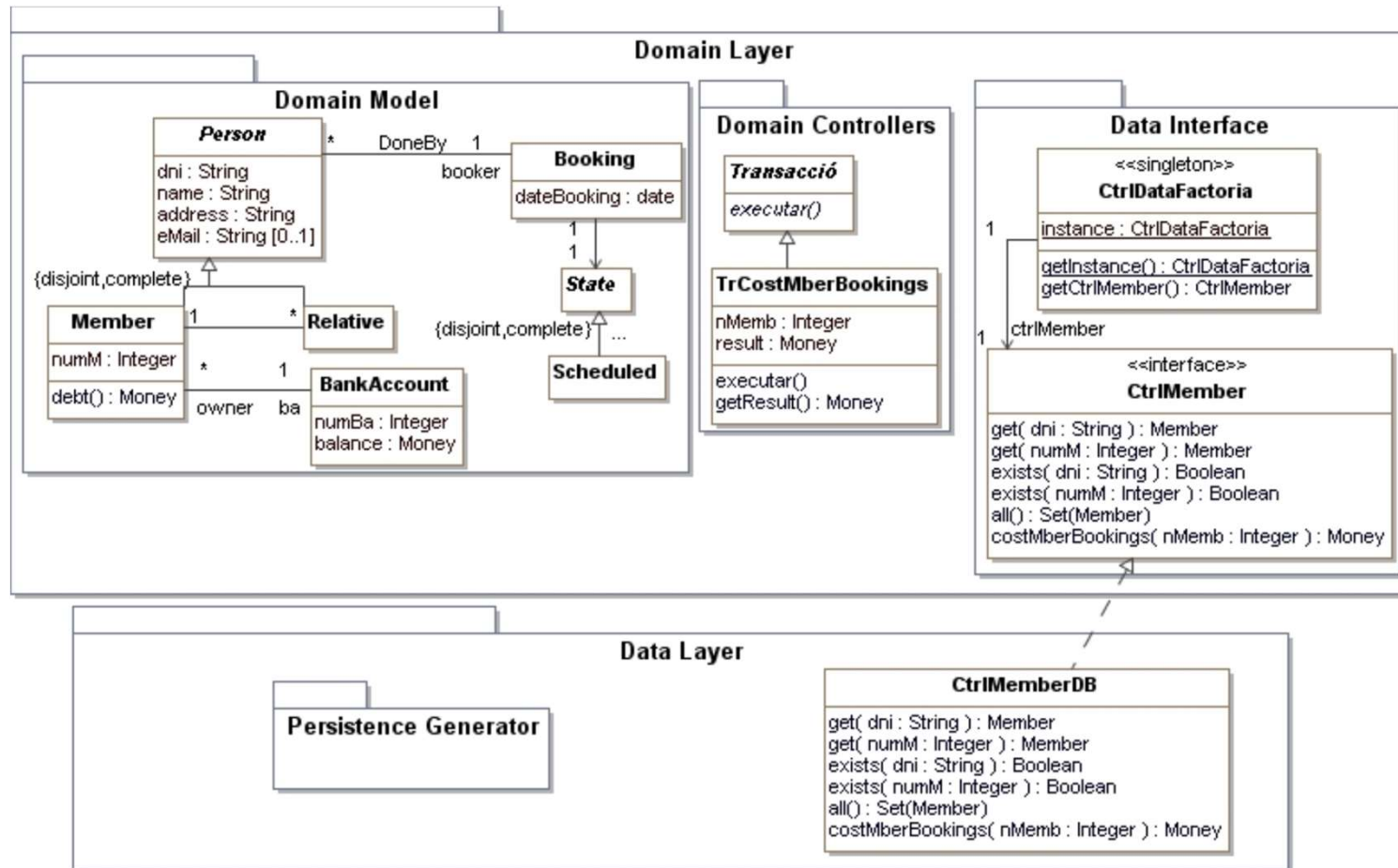
- The automatic generation of persistence is very comfortable for updating the DB but not so much for certain queries
 - Cumbersome and error-prone sequence diagrams
 - Potential loss of efficiency
- Option: the Data layer offers those query operations required by the problem at hand
 - The Domain layer acts just as an intermediary
- Current generators of persistence use to offer this possibility:
 - Ex: Hibernate, *createQuery*

Automatic generation of persistence:

Arbitrary query operations on the Data layer, example

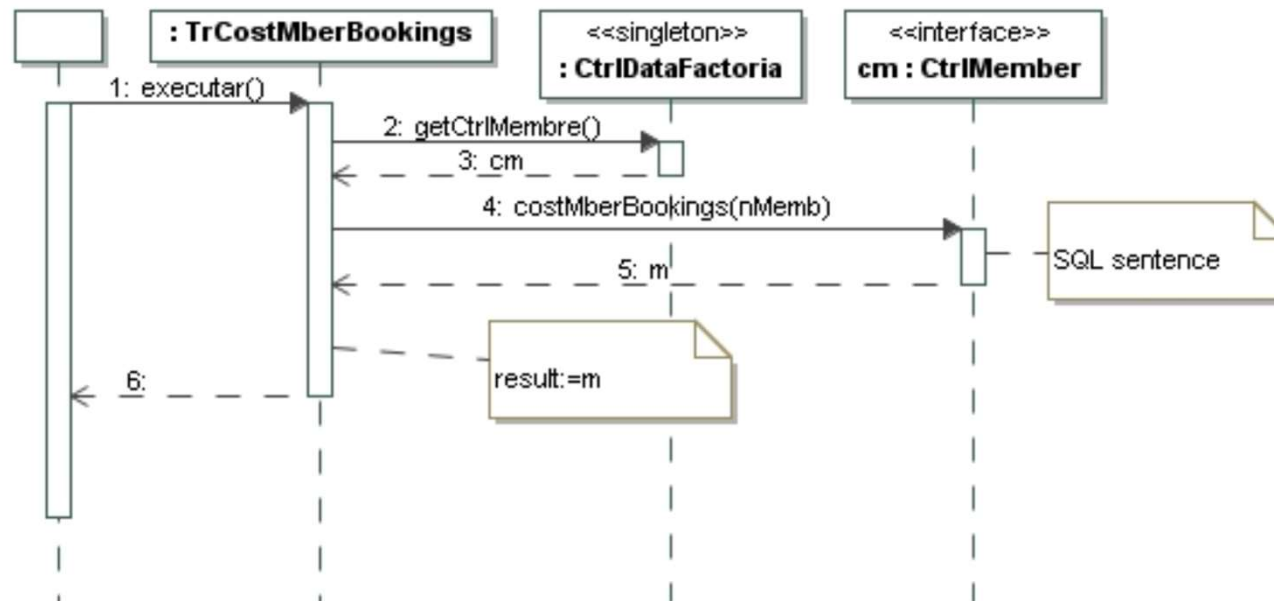


Automatic generation of persistence: Arbitrary query operations on the Data layer, example

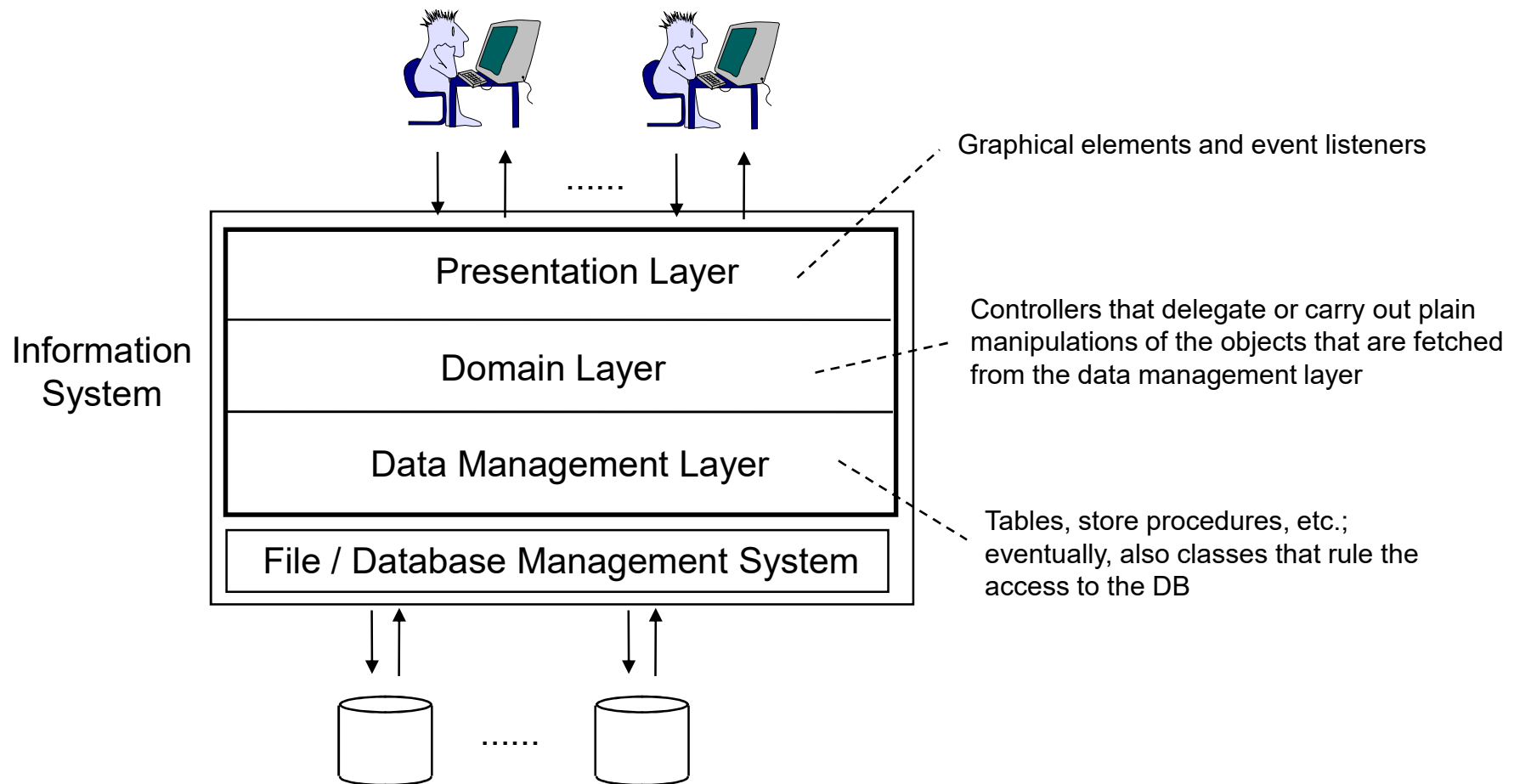


Automatic generation of persistence:

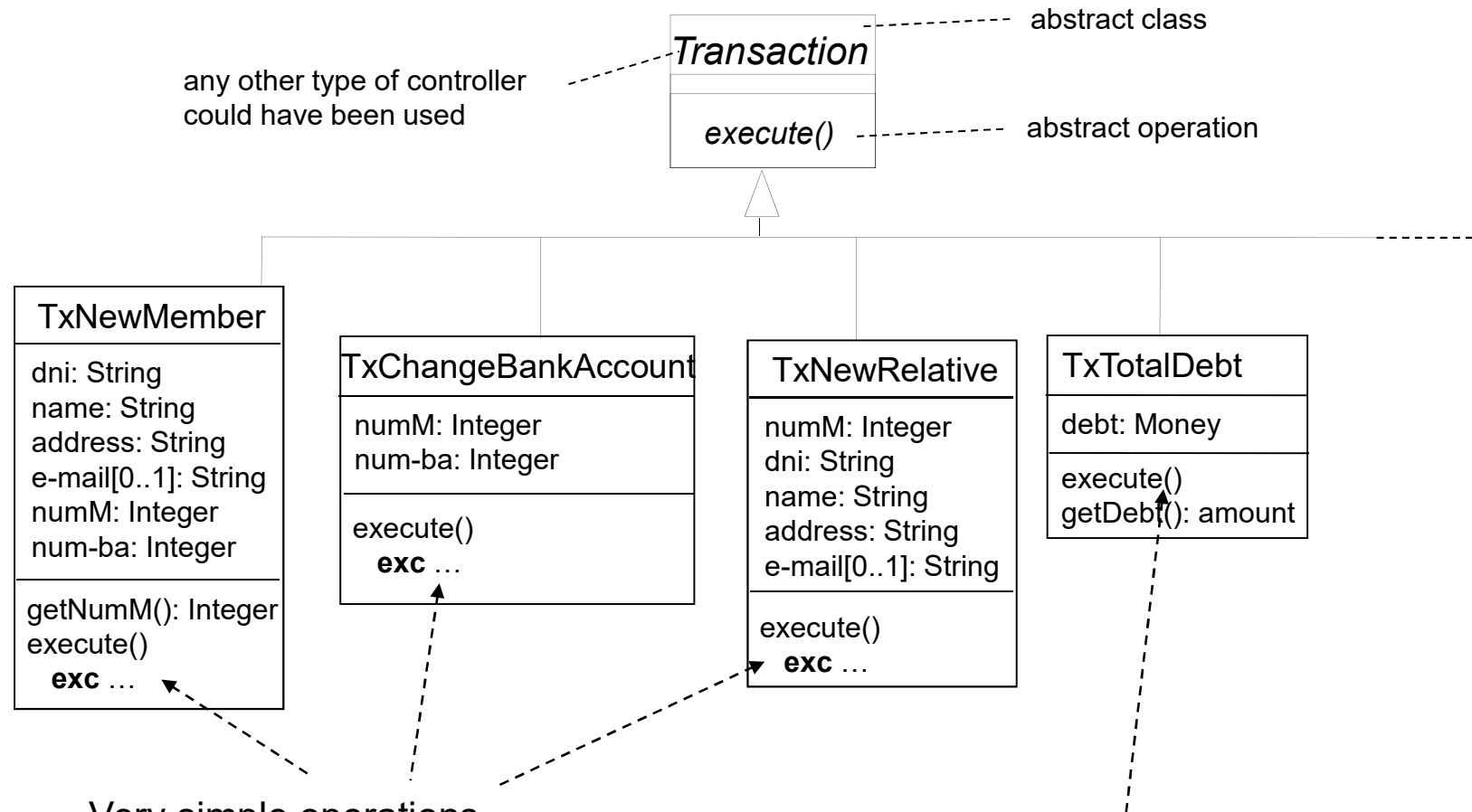
Arbitrary query operations on the Data layer, example



Direct design of persistence: Layer configuration



Direct design of persistence: Design of operations



Very simple operations

The objects of the diagram (Member, Relative, ...) do not get any responsibility assigned
They access the DB through calls to other operations

Direct design of persistence:

Design of operations, strategies

- Domain layer controller with plain logic
 - The controller calls operations that access the DB for obtaining and updating data
 - ✓ Use of data access patterns
 - The controller performs simple manipulations over these data
 - Domain layer controller as a pure intermediary
 - The controller delegates on a stored procedure that takes on all the responsibility
- It is possible to combine both strategies
- In both cases, it is also possible to assign some responsibilities to SQL elements directly as done previously

Direct design of persistence:

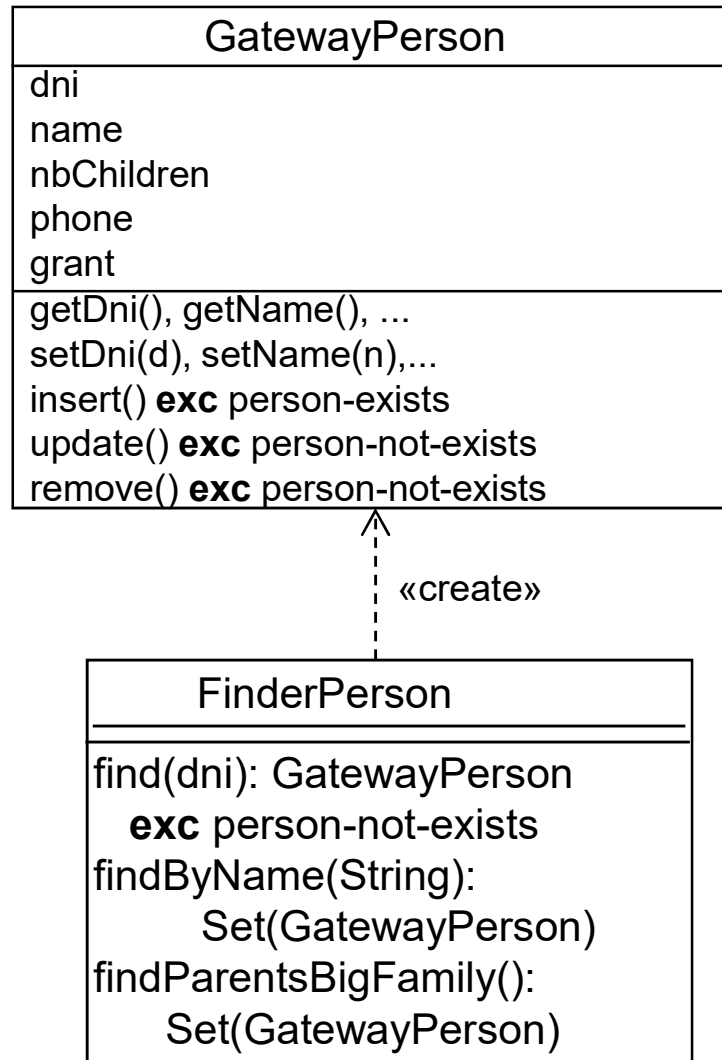
Domain layer controller with plain logic

Patterns of the Data layer for direct design in relational DB

- Table Data Gateway
 - A singleton class for each data base table
 - Encapsulates the access (look-up, update) to a table
- Row Data Gateway
 - A class for each data base table. An instance corresponds to a table row
 - The look-up operations may be class operations or be defined in another class
 - Encapsulates the update of a row.
 - It may be applied to simple designs.
- Active Record
 - Row data gateway with operations that come from the problem domain

Direct design of persistence:

Row data gateway



- A gateway class for each table (or view). Instances correspond to table rows. Each column in the table is an attribute of the class
- Finder operations access the table through SQL sentences (or stored procedures) and only may create gateway instances with the result
- The *insert*, *update* and *remove* operations contain SQL sentences (or stored procedures)

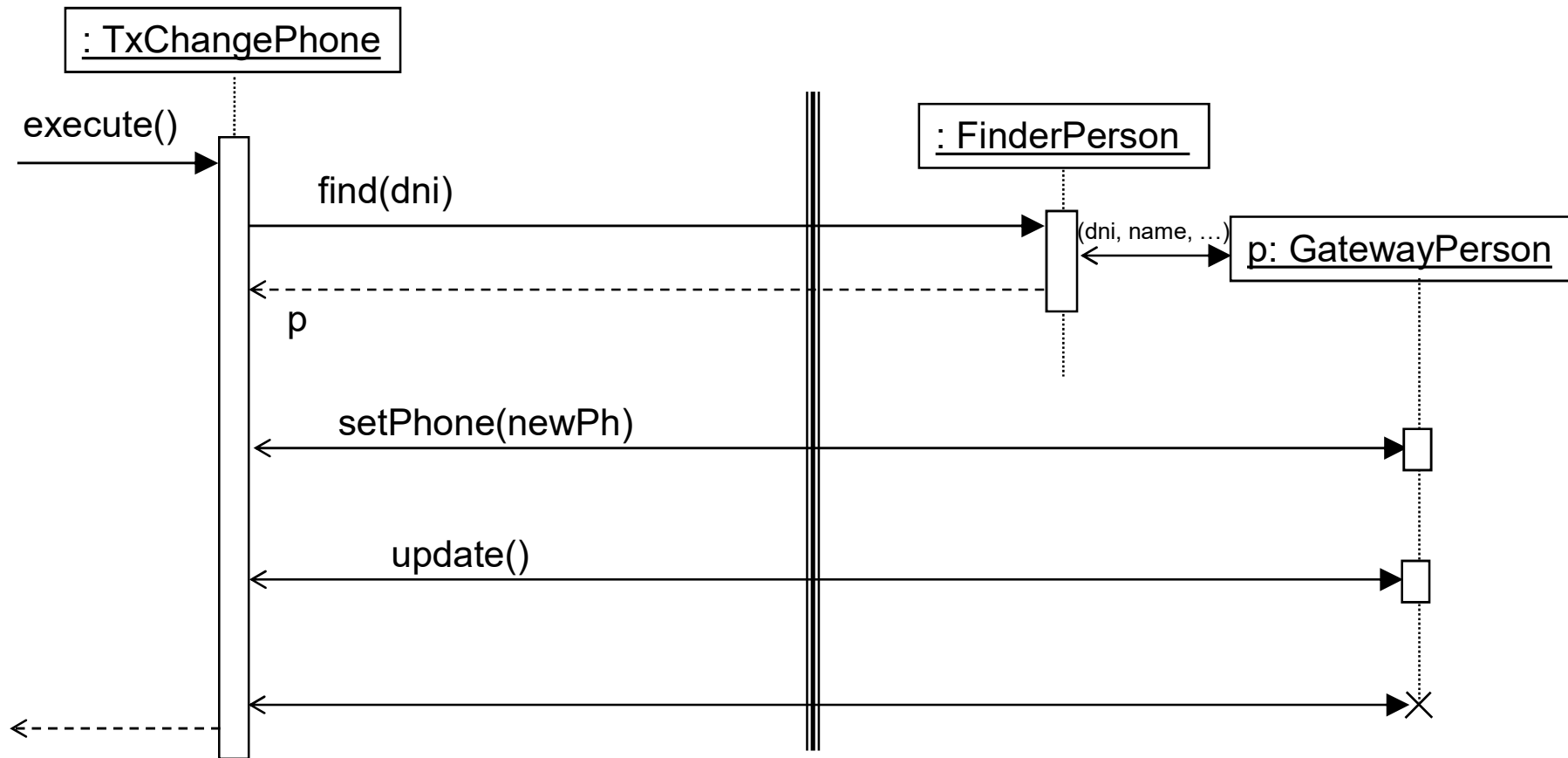
Direct design of persistence:

Design using the row data gateway

- The design of the operations may exploit the power of SQL
- Domain layer controllers:
 - Create gateways for inserting objects
 - Ask finders for obtaining objects (gateways)
 - Manipulate these objects in main memory
 - Pull the objects back to the Data layer using the gateway

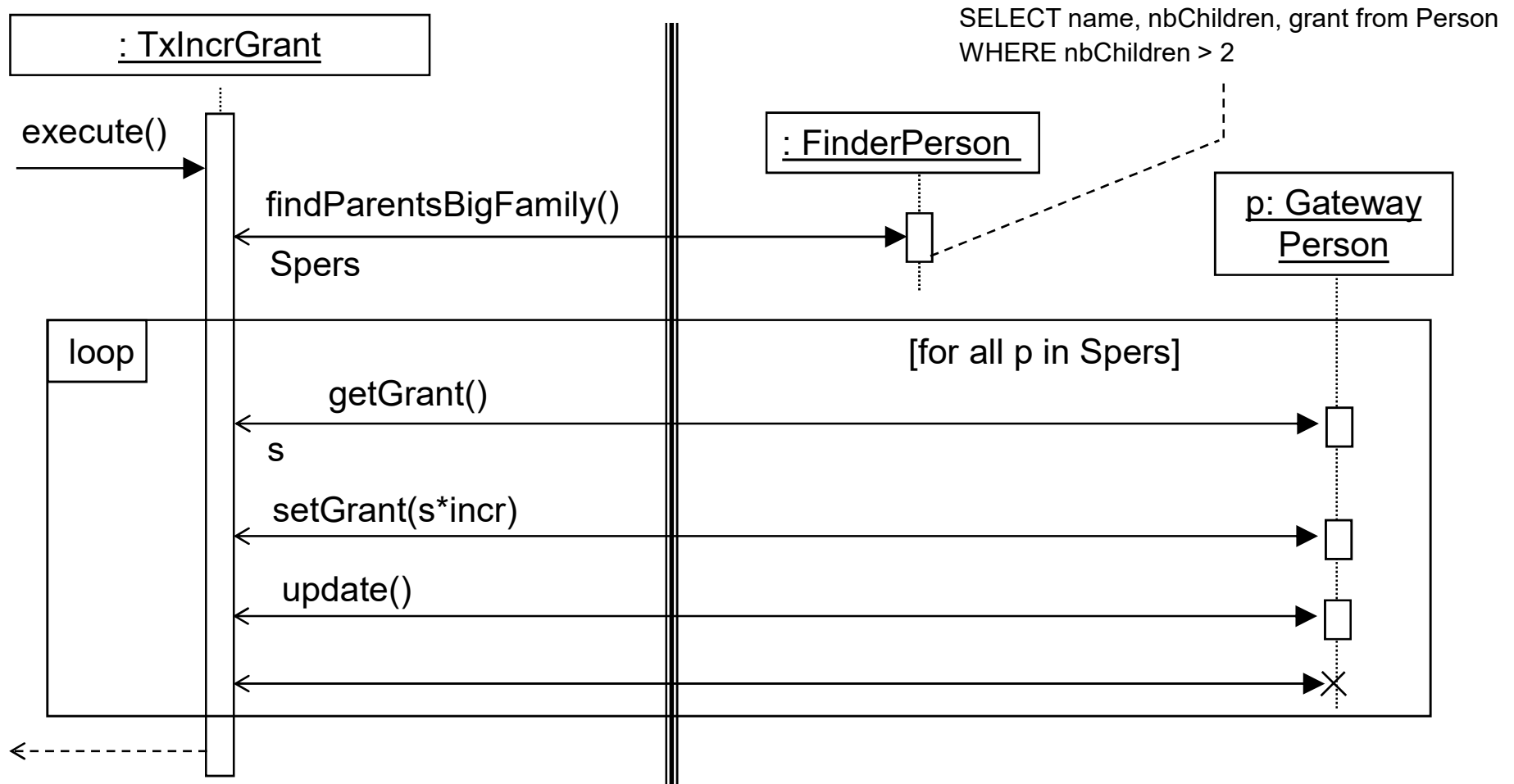
Direct design of persistence:

Design using the row data gateway, example



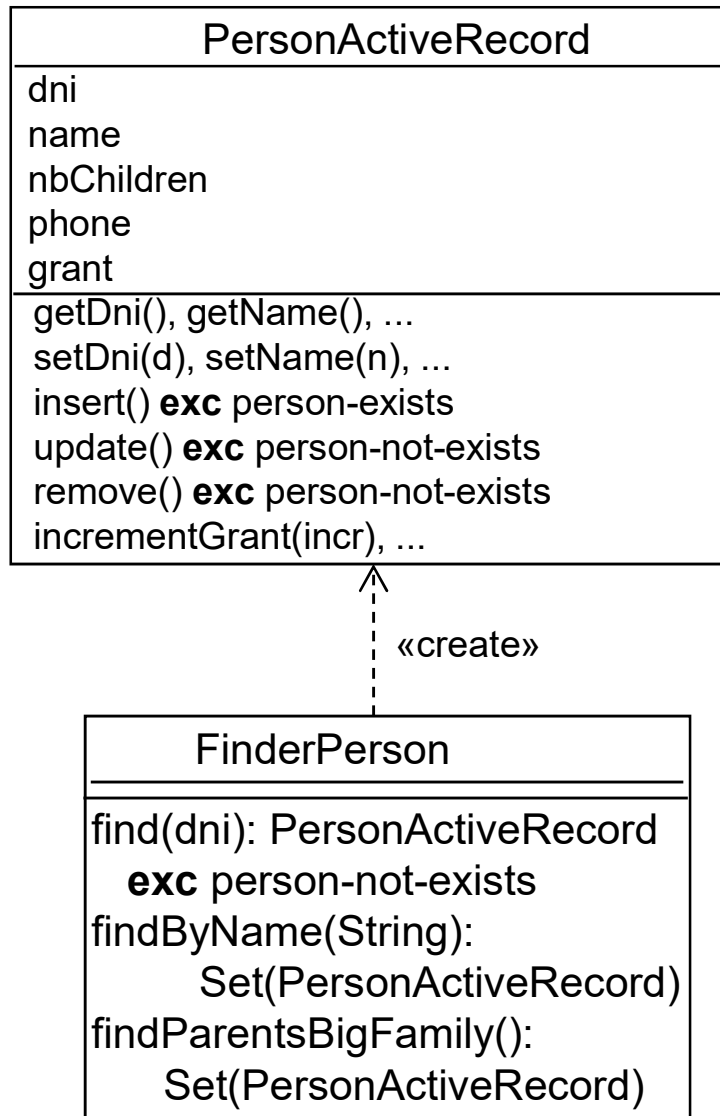
Direct design of persistence:

Design using the row data gateway, example



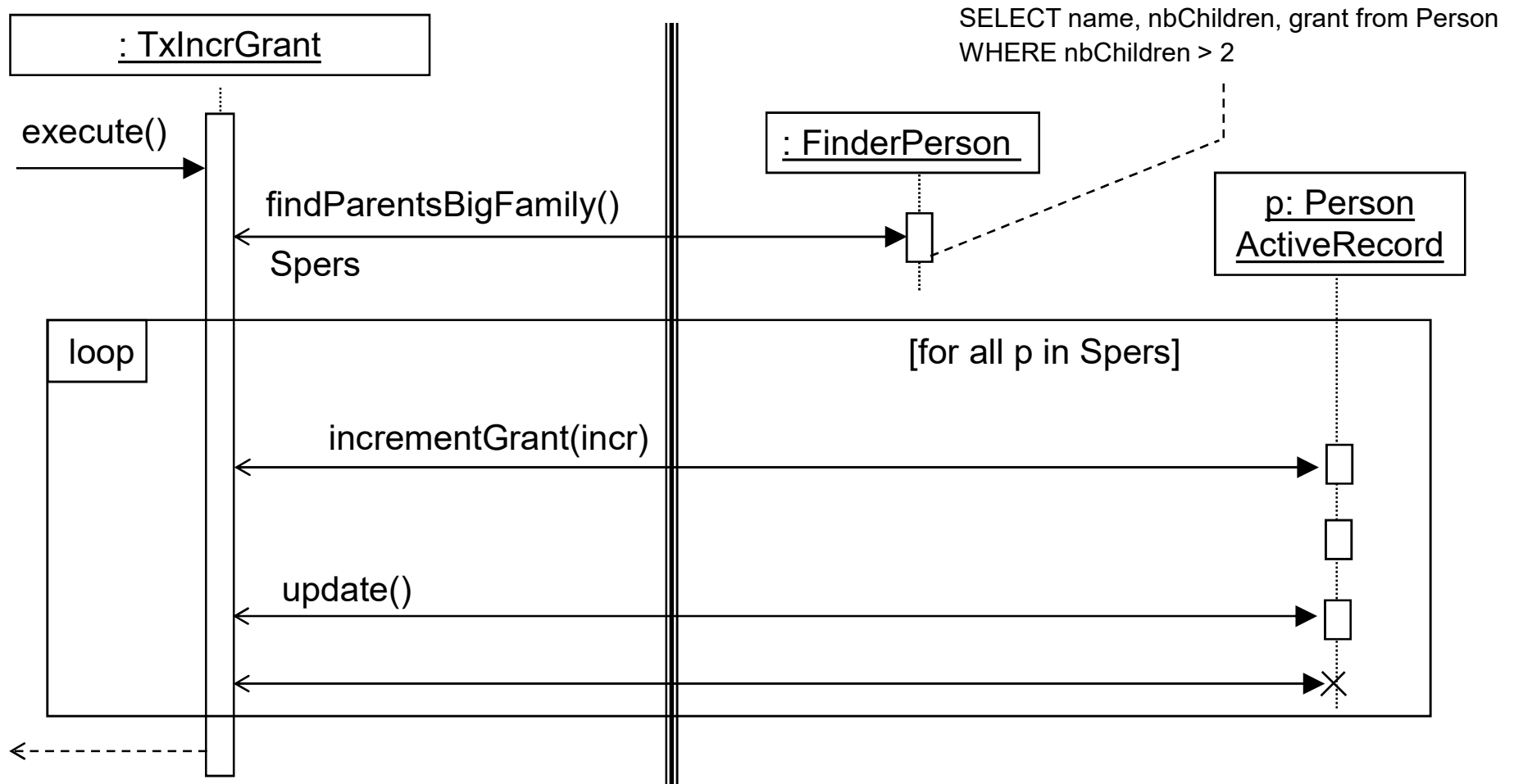
Direct design of persistence:

Active Record



- An active record class for each table (or view). Similar to row gateway pattern
- Finder operations similar to row gateway pattern
- The *insert*, *update* and *remove* operations similar to row gateway pattern
- Implement some pieces of business logic

Direct design of persistence: Design using the active record, example



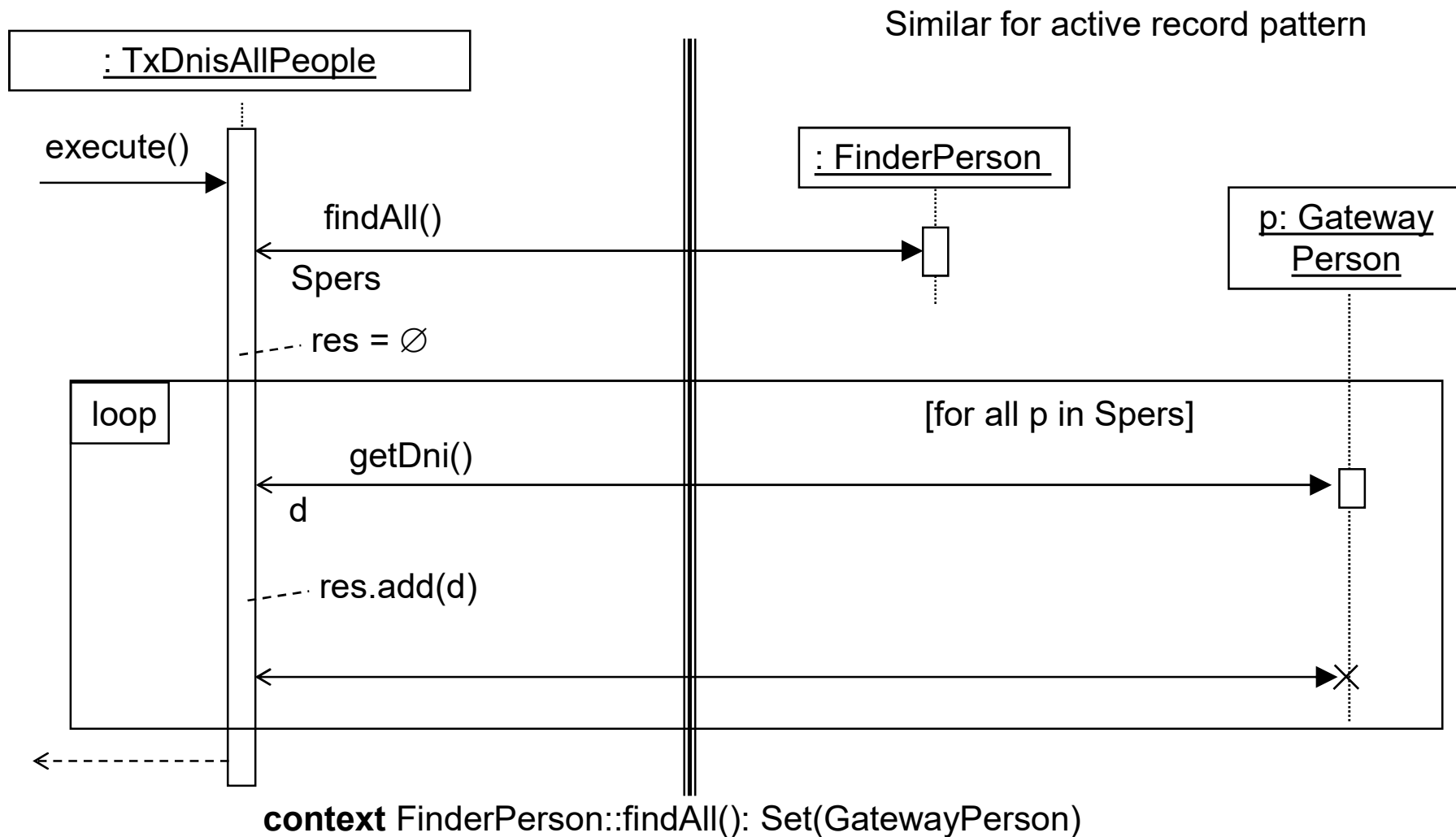
Direct design of persistence:

Design using the row data gateway and active record with arbitrary query operations

- As it happened with the Domain Model pattern, some domain query operations may be cumbersome using the pattern strictly since
 - Restricted to the table scope
 - Just gateways allowed as result
- Solution: allow query operations in Finders
 - Collection retrieval
 - Computation of results

Direct design of persistence:

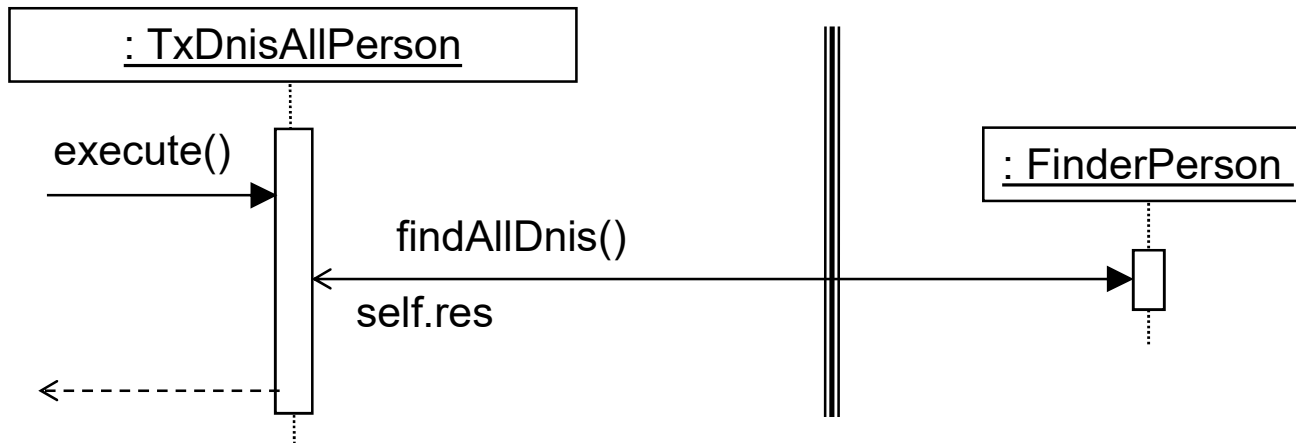
Design using the row data gateway, example with simple query operations



Direct design of persistence:

Design using the row data gateway and active record, example with arbitrary query operations

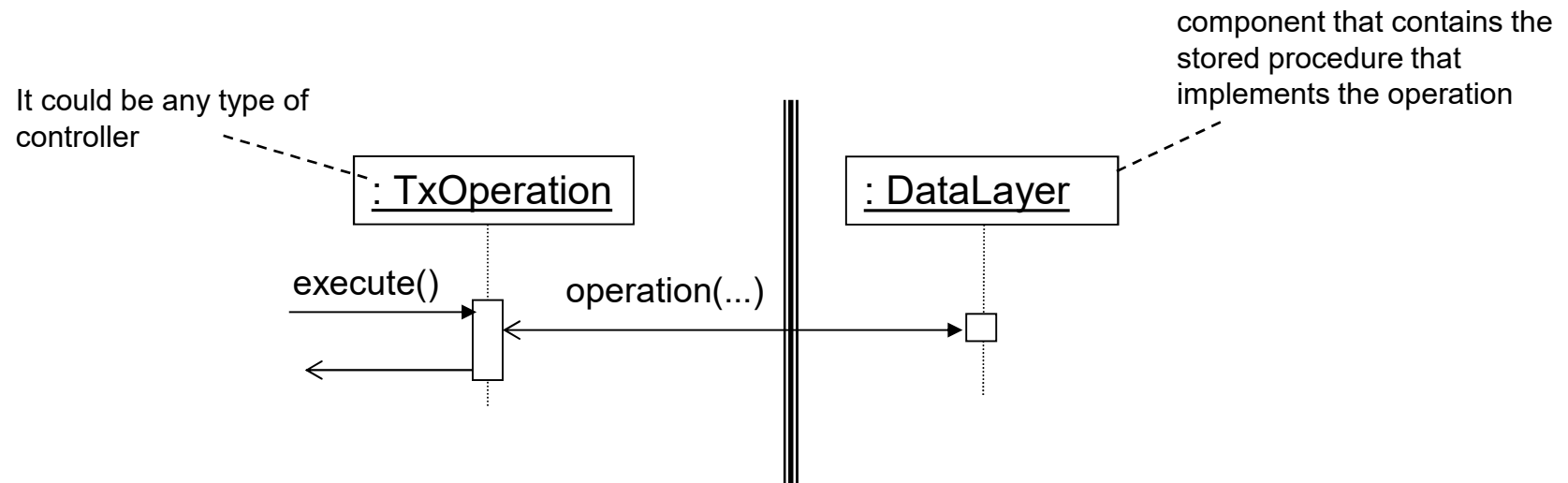
Similar for active record pattern



context FinderPerson::findAllDnis(): Set(String)

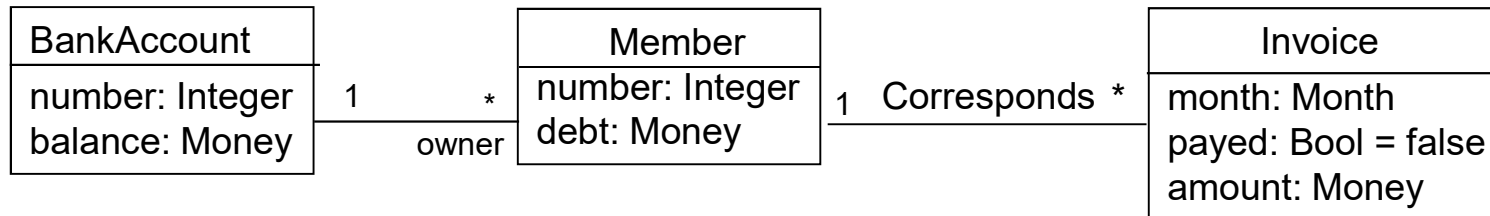
Direct design of persistence:

Domain layer controller as pure intermediary



Example of persistence management:

Problem statement



context DomainLayer::payInvoice (ns: Integer, m: Month)

-- operation for paying the Invoice of month *m* from Member *ns*

exc invoice-not-exists: it does not exist an Invoice from Member *ns* in month *n*

invoice-payad: the invoice is already payad

low-balance: the member has not balance enough in its bank account

post:

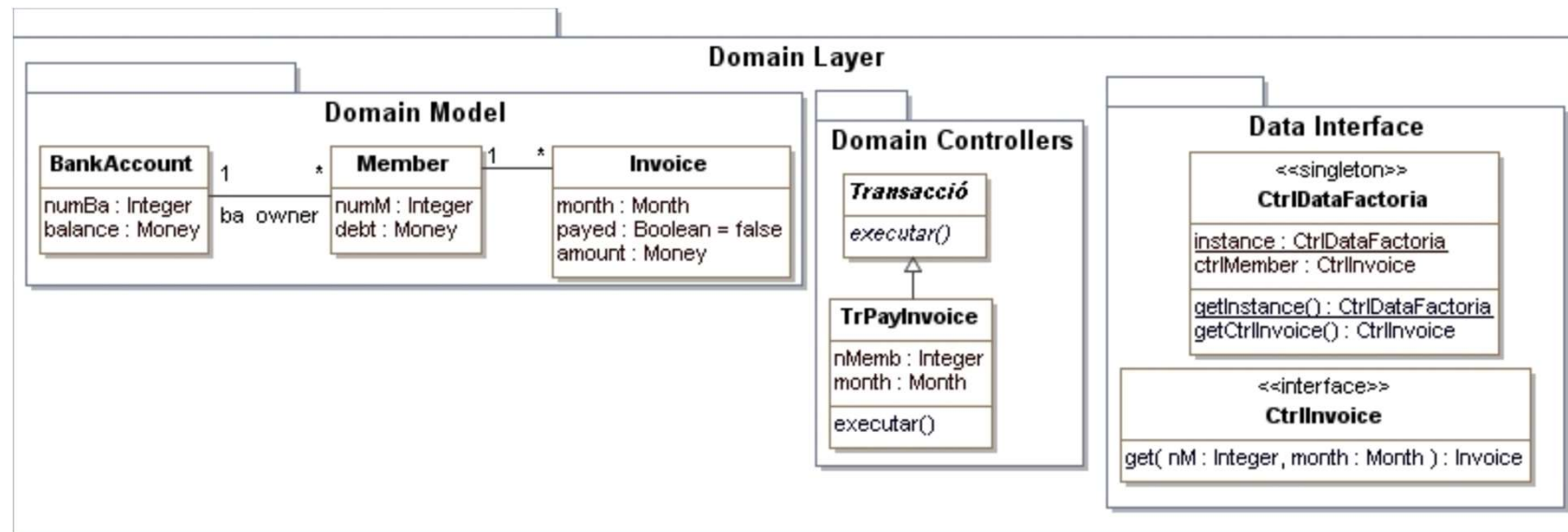
2.1 The attribute *payad* from *Invoice* is set to true

2.2 Balance of Member's BankAccount is decremented with Invoice's amount

2.3 Member's debt is decremented with Invoice's amount

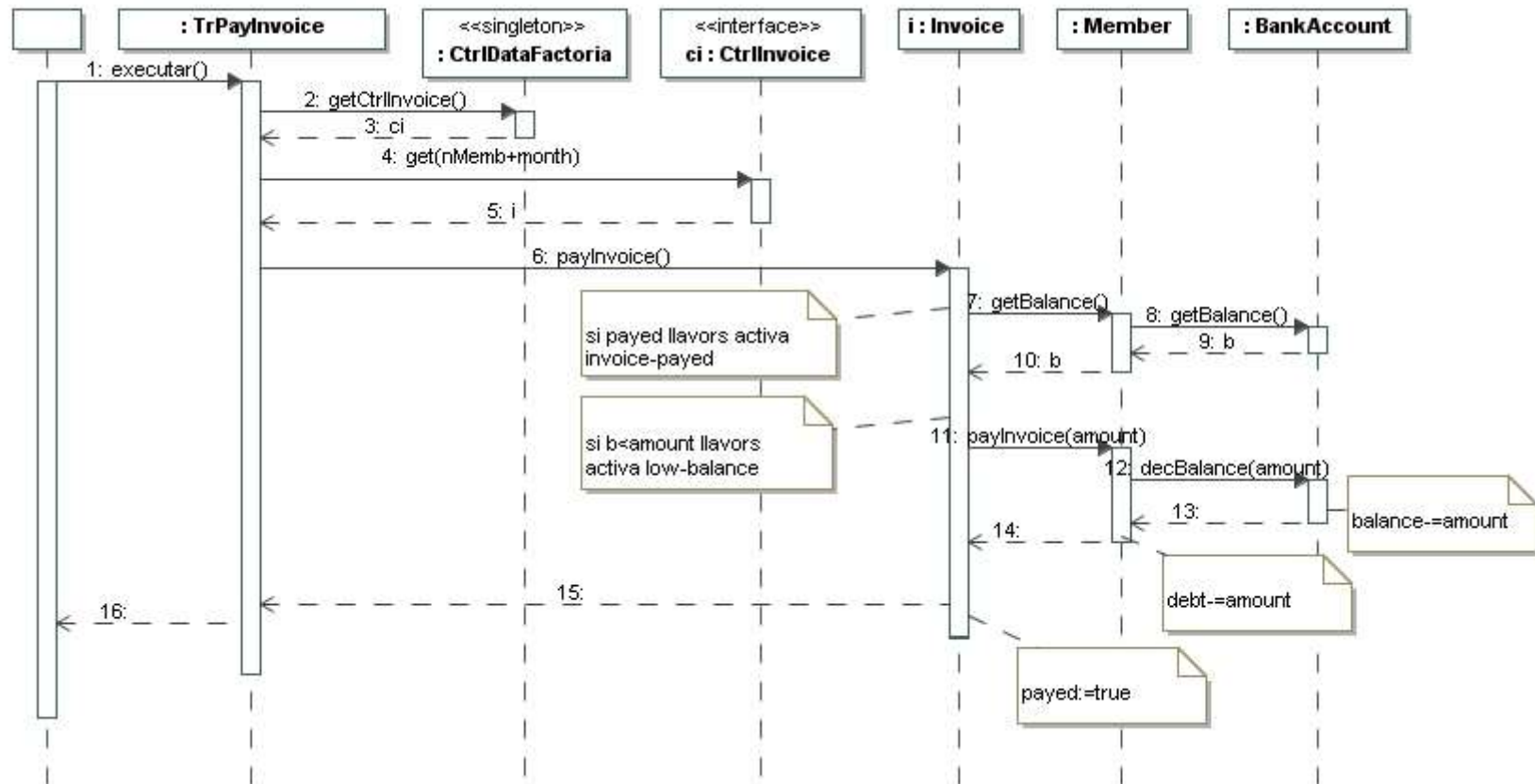
Example of persistence management:

Solution without considering persistence



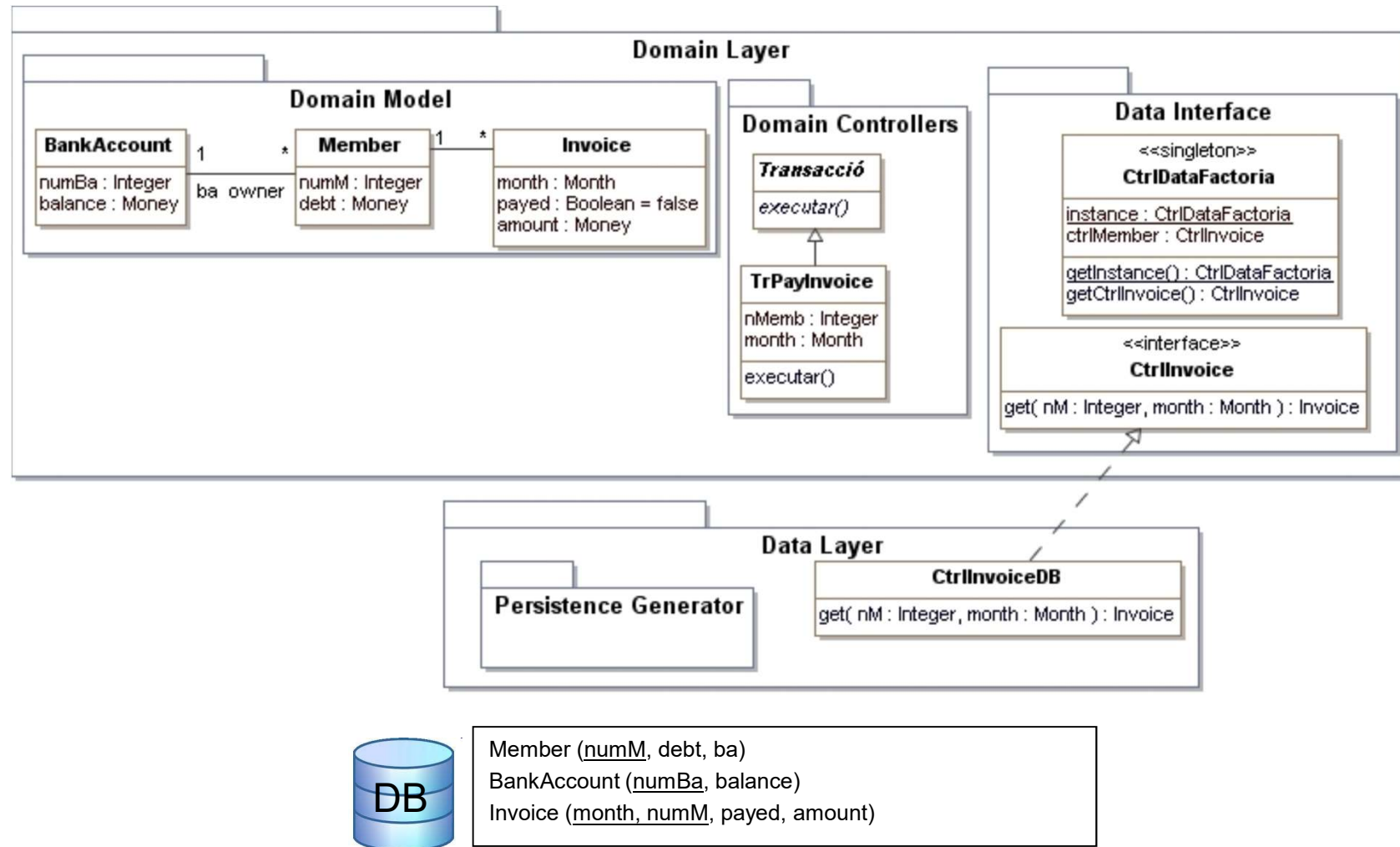
Example of persistence management:

Solution without considering persistence



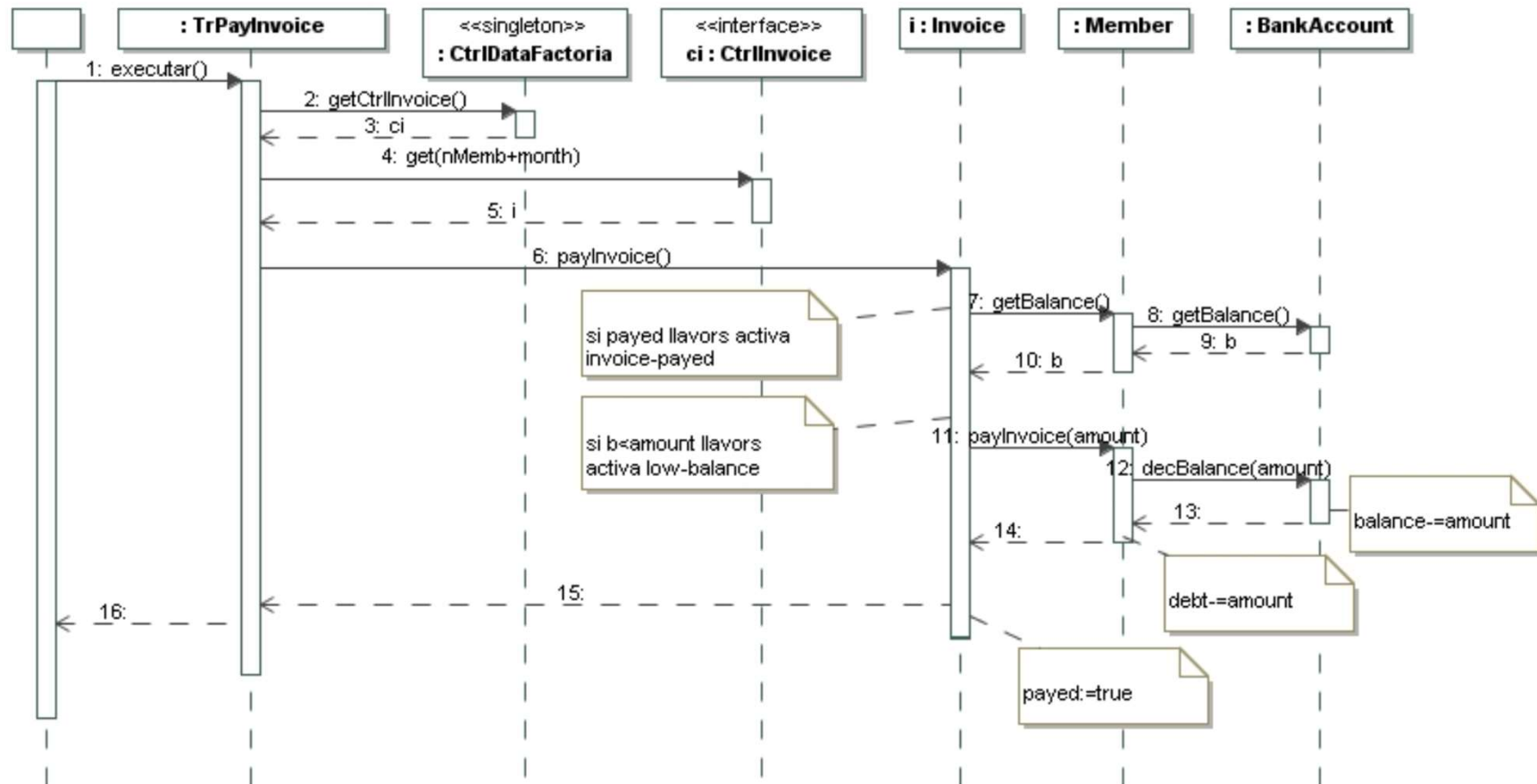
Example of persistence management:

Solution with automatic persistence management



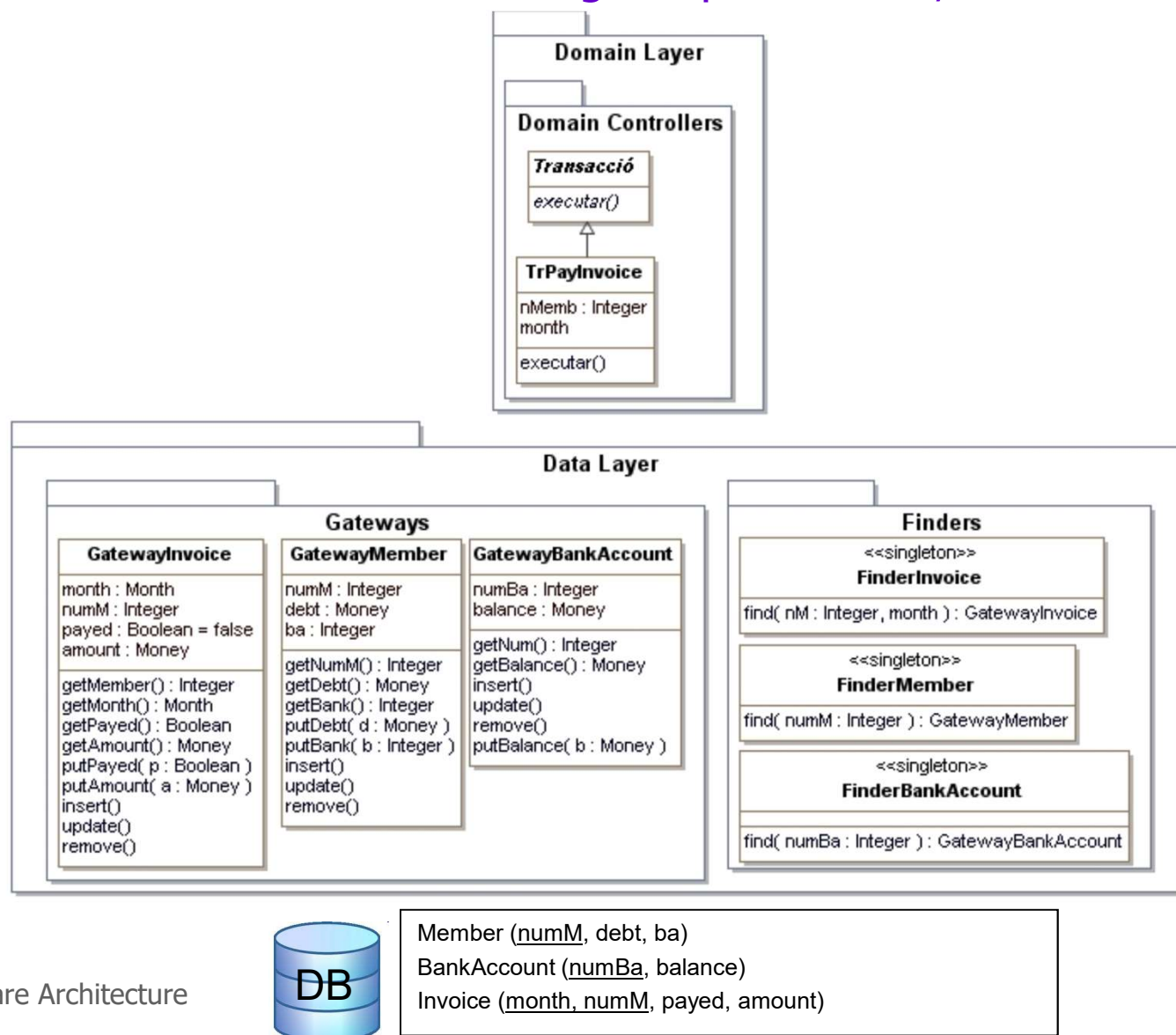
Example of persistence management:

Solution with automatic persistence management



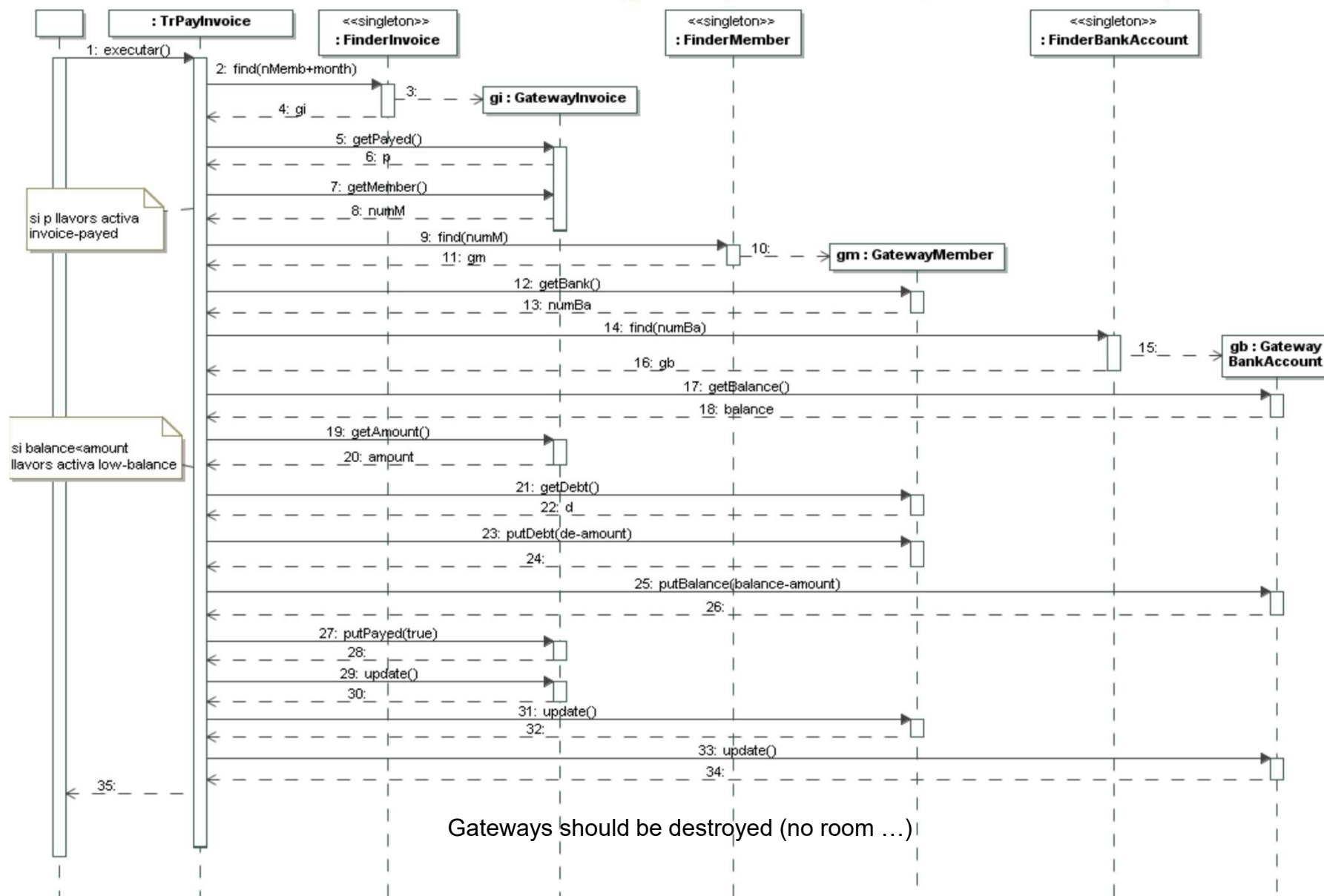
Example of persistence management:

Solution with direct design of persistence, Row Gateway



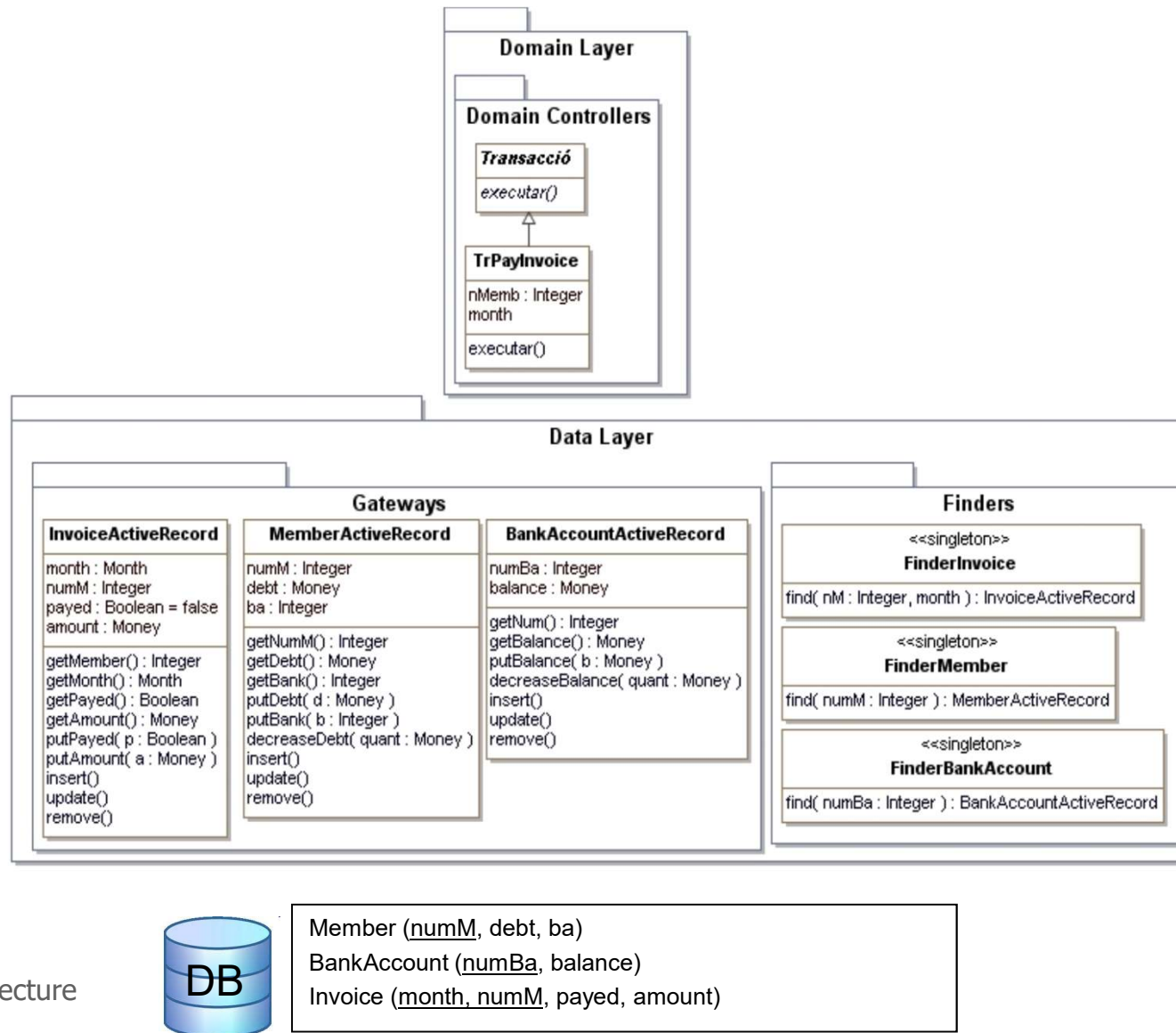
Example of persistence management:

Solution with direct design of persistence, Row Gateway



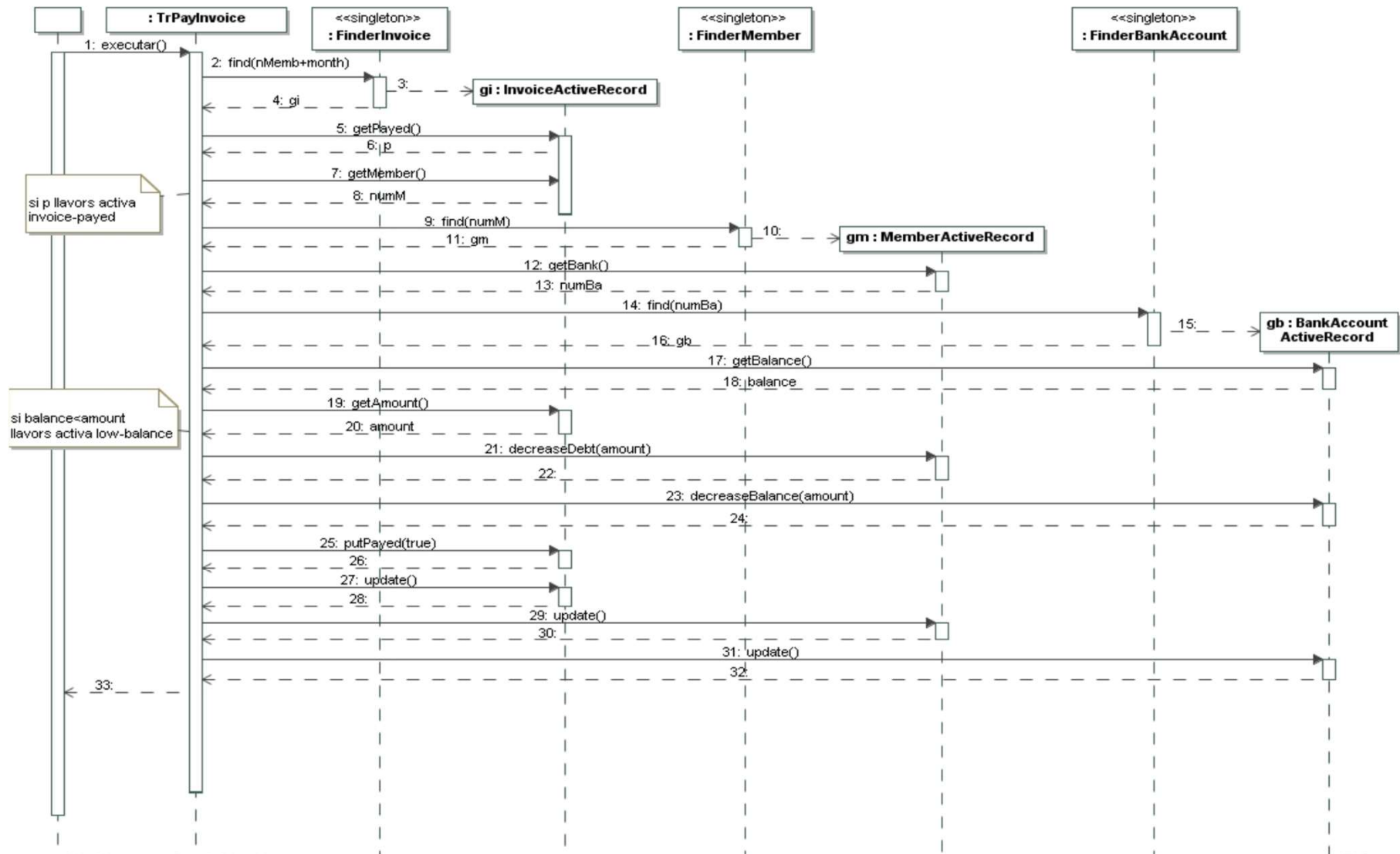
Example of persistence management:

Solution with direct design of persistence, Active Record



Example of persistence management:

Solution with direct design of persistence, Active Record



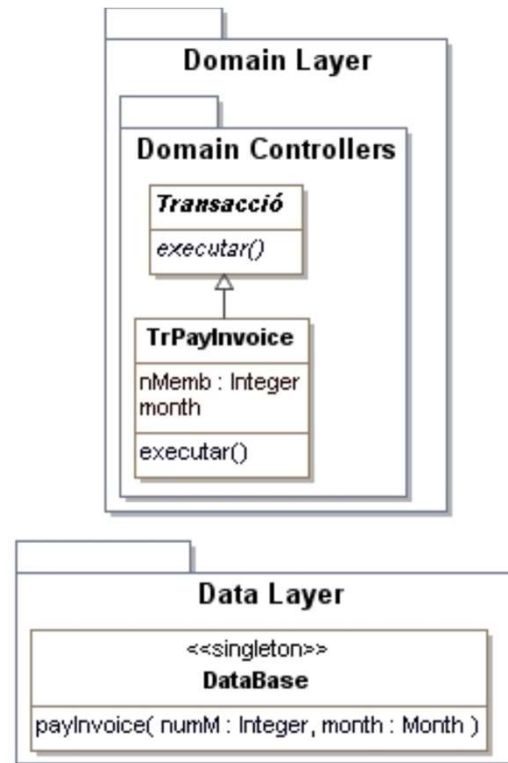
Software Architecture

Gateways should be destroyed (no room ...)

61

Example of persistence management:

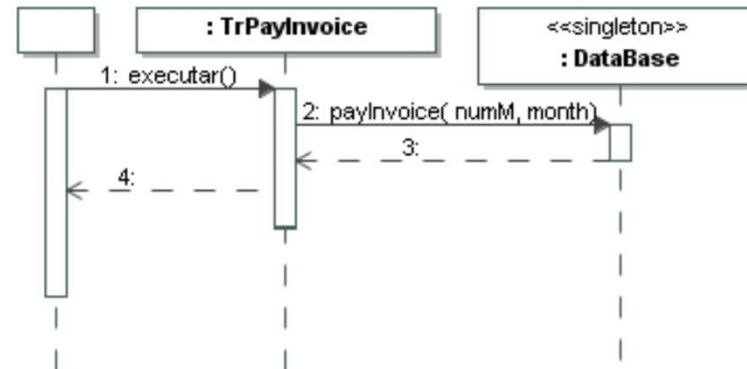
Solution with direct design of persistence, Stored Procedures



Member (numM, debt, ba)
BankAccount (numBa, balance)
Invoice (month, numM, payed, amount)

Example of persistence management:

Solution with direct design of persistence, Stored Procedures



```

CREATE PROCEDURE PayInvoice (numMemb, monthTramitation)
RETURNING INTEGER, CHAR(50);
DEFINE error-code INTEGER; ....
ON EXCEPTION SET error-code, error-miss; RETURN error-code, error-miss END EXCEPTION;

IF ((SELECT COUNT(*) FROM invoice WHERE month=monthTramitation AND member=numMemb)=1) THEN
  LET amount, l-paid = (SELECT amount, paid FROM invoice WHERE month=monthTramitation AND member=numMemb));
  IF ('Y' = l-paid ) THEN RAISE EXCEPTION 2, 'The invoice is already payed';
  ELIF LET balance, numba = (SELECT c.balance,c.num-ba FROM member s, bankaccount c WHERE s.ba=c.num-ba
                                                                    and s.numM=numMemb);
    IF balance < amount THEN RAISE EXCEPTION 3, 'The member has not balance enough';
    ELSE UPDATE invoice SET paid = 'Y' WHERE month=monthTramitation AND member=numMemb;
        UPDATE member s SET s.debt = s.debt-amount WHERE s.numM=numMemb;
        UPDATE bankaccount c SET c.balance = c.balance-balance WHERE num-ba =c.num-ba ENDIF
ELSE RAISE EXCEPTION 1, 'The member has not invoices this month'; END IF;
RETURN 0,'PayedInvoice';
END PROCEDURE;
  
```

Design of Domain and Data layers, summary

- Domain and Data layer patterns are related:
 - Domain Model + Data Mapper
 - variant: Domain Model with arbitrary queries at the Data layer
 - Transaction Script: supports two variants
 - Transaction Script + (Row Gateway o Active Record)
 - Variant: Row Gateway o Active Record with arbitrary queries
 - Transaction Script + Stored Procedures
 - May be combined
- In all the cases, it is possible to assign directly to the DB schema responsibilities coming from the layer responsibility assignment :
 - Coming from the OO data conceptual model (e.g., keys)
 - Coming from the contracts themselves (e.g., delete on cascade)
 - In the case of Data Mapper, it depends on the capabilities of the employed technology

References

- M. Fowler
Patterns of Enterprise Application Architecture
Addison-Wesley, 2003
- H. Garcia-Molina, J. Ullman, J. Widom
Database Systems Implementation
Prentice-Hall, 2000.
- J. Melton, A. Eisenberg
Understanding SQL and Java Together
Morgan-Kaufmann, 2000.
- Christian Bauer, Gavin King
Hibernate in Action
Manning Publications Company, 2004